A combinatory formulation of PTS-rk

October 24, 2026

Abstract

PTS-rk admits a complete Hilbert presentation by finitely many numerically indexed axiom schemes, by the supersorted-system theorem of Bunder and Dekkers. A finite basis of fixed closed typed combinators is nevertheless impossible: application cannot increase the rank of its function's type. We give a rank-typed SK interface, an explicit Hilbert catalogue and a first-order algebraic presentation using quotient inductive-inductive-recursive syntax. Checked soundness and extraction are distinguished from the remaining formal completeness and adequacy proofs. All expansions retain the original annotations and use beta conversion only. The simply typed surface has canonical SK outputs for native beta equality, a quotient isomorphism and typed normalizing application. A normal-form algebra covers original dependent typing, with normalized substitution and computed types; a structural checker decides the original typing judgment in well-formed contexts. These results are not yet an equivalence with the combinatory QIIRT. Elementary bounds control native normalization and every intermediate term; fixing the input's universe labels gives finite syntax carriers and a total, typed Agda apply-and-normalize function. Independent certificate checking gives contextual soundness in graded finite carriers and external exclusion of a closed proof of Void. Finite Church encodings give explicit rank-bounded code-to-code reduction programs, separated from the proposed internal consistency proof. A ground-query Horn reduction with explicit Void denials excludes contradictory models. Finite ground fragments have exact, computable models; signed completion reflects all formulas in their finite atom language. Quantified MM0 fragments have a complete fixed-domain constructor search and proof-producing finite satisfaction certificates. The literal MM0 theory agrees with the independent catalogue; all size/depth-guarded schemas have finite models with empty Void. Each fixed finite fragment has an independent Church proof encoding and an internal soundness function for internally quantified queries and arbitrary proof inhabitants, with proof-type rank at most three and soundness-function type rank at most four. One finite, ungraded carrier cannot, however, support all original PTS typings and conversion while keeping Void empty.

1 Introduction and goals

This note develops the programme of a second paper on PTS-rk: combinatory presentations, bounded models and internal interpretation. Its intended deliverables are the following six goals. They are research targets, not a claim that the constructions below already establish all of them.

1. **A Church-encodable natural-model QIIRT.** Give an intrinsically typed, binder-free presentation of the full original PTS-rk, with contexts, substitutions, types and terms in the natural-model style. The first principal Agda source should specify its constructors, computed typing and equations, prove correspondence with original PTS-rk, and admit a Church realization in PTS-rk. Encoding the quotient equations and their elimination interface is part of the goal, not a consequence of merely encoding raw syntax.

2. **A normal-form-first presentation.** Give a second intrinsically typed, binder-free (Q)IIRT with recursive dependent apply-and-normalize. The target is that beta-equal closed terms and types compute to definitionally equal canonical representations, including their normalized type indices. Proof witnesses must not obstruct this computational

agreement. Prove the general soundness, completeness and termination statements separately; a propositional equality theorem alone does not meet the definitional target. This unrestricted normalizing model is not intended as one fixed-rank internal evaluator. The anticipated obstruction must connect the TOWER lower bound to the chosen encoding and cost model [8]; it is not inferred from the complexity label alone.

3. **A cumulative hierarchy of bounded fragments.** Give a third Agda source defining indexed models of the kind in (2), bounded by length and by every relevant rank occurring inside terms, types, annotations and declarations, not only the outer rank. Bound universe labels as well. Prove coherent inclusions, coverage of every finite input, and application into an explicitly computed larger grade. At each fixed grade, Church encode the carriers, apply-and-normalize and extraction into native PTS-rk terms, with explicit ranks and correctness laws. Cumulativity here means inclusions of fragments, not new universe-cumulativity rules for PTS-rk.

4. **A Hilbert-style logical theory.** Present PTS-rk by indexed logical axioms and inference rules, explain terms as proofs, and establish original-PTS soundness and completeness. Formalize the literal theory and its correspondence proofs in Metamath Zero, not only example derivations of an independently declared specification.

5. **Partial finite models and internal bounded soundness.** Construct partial models of finite fragments with empty Void, and use their PTS-rk realizations to establish bounded soundness inside PTS-rk. The ultimate target is an internal statement expressing that no closed proof of $\mathsf{Void} = \Pi X : U_0.X$ exists, with its construction verified in MM0. The proof predicate, represented domain and internal consistency sentence must be fixed explicitly. Results for every external fixed bound do not by themselves provide one internal proof covering all bounds; the precise target and remaining transfer obligations are in Section 7.7.

6. **An indexed self-interpreter with source folds.** Construct PTS-rk interpreters in PTS-rk, indexed by the required ranks and resources, with quotation, native extraction and folds over source code. A typed CPS translation should exemplify a non-evaluation fold. Retain intensional source codes when needed: a fold defined on a beta quotient must instead be proved to respect every quotient equation.

The three Agda presentations should share one original typing/equality contract and explicit adequacy maps. "Binder-free" excludes object-level lambda binders from their computational syntax; it does not exclude Agda functions, contexts, or a codomain in an extended context. Merely returning combinators after normalization through native binders does not yet meet the direct binder-free algorithmic goal.

Development order. First fix the common interfaces, resource indices and encoding contracts, then complete the natural-model and Hilbert/MM0 correspondences. Test closed definitional computation in a small dependent normal-first prototype before consolidating the second source. Next derive the bounded hierarchy, Church compilation and native extraction; use these to internalize bounded soundness and indexed interpretation/CPS. Finally address the separate all-budget consistency obligation. Each stage requires safe Agda checking and the stated adequacy proofs, not only computational examples. The detailed milestones and acceptance criteria are recorded in the second-paper plan[1]. Existing proofs below are reusable components; new construction is deferred until this plan is resumed.

[1] `SECOND-PAPER-PLAN.md`

1.1 The calculus and its logical reading

Terms of PTS-rk are variables, universes U_i ($i \in \mathbb{N}$), products $\Pi x : A.B$, annotated abstractions $\lambda x : A.t$, and applications tu. Contexts are lists of typed variables. Its distinctive rules are

$$\Gamma \vdash U_i : U_0, \qquad \frac{\Gamma \vdash A : U_a \quad \Gamma, x : A \vdash B : U_b}{\Gamma \vdash \Pi x : A.B : U_{r(a,b)}}, \qquad r(a,b) = \max(a+1, b).$$

Typing and conversion are those of the original calculus: annotations are retained, with neither cumulativity nor eta conversion. In particular, $U_i : U_0$ is not the usual ascending universe rule $U_i : U_{i+1}$.

A logical judgment $\Gamma \vdash t : A$ reads: under assumptions Γ, t is a proof of the proposition represented by A. Types may also describe data; there is no separate primitive universe of propositions. A product $\Pi x : A.B(x)$ is universal quantification over inputs $x : A$; when the input and output are propositions and B does not depend on x, it is implication $A \to B$. Quantification over $A : U_a$ allows types and propositions themselves as parameters.

Application is dependent elimination:

$$\frac{\Gamma \vdash f : \Pi x : A.B(x) \quad \Gamma \vdash u : A}{\Gamma \vdash fu : B(u)}.$$

For $A \to B$, this is modus ponens, with fu recording the proofs used. Natural deduction discharges assumptions by abstraction; a Hilbert system encodes discharge through closed axioms and application. A derivation certifies the judgment, while its subject term is the object-level proof. Beta reduction computes within that proof without changing its type.

2 The rank-typed SK interface

The surface syntax has two combinators, with independently instantiated type parameters at every occurrence:

$$\begin{aligned} &K : X \to Y \to X, && Kxy \longrightarrow x, \\ &S : (X \to Y \to Z) \to (X \to Y) \to X \to Z, && Sfgx \longrightarrow fx(gx). \end{aligned}$$

Expansion replaces these heads by their annotated lambda definitions. Ranks use $r(a,b) = \max(a+1, b)$. Identity is the three-atom abbreviation $\mathsf{id}_X = S_{X,Y\to X,X} K_{X,Y\to X} K_{X,Y}$, not a primitive; one may choose $Y = X$. Indices are suppressed below, never discarded by the formal syntax.

Bracket abstraction eliminates a variable by

$$[x]x = \mathsf{id}, \qquad [x]a = Ka \ (a \neq x \text{ a leaf}), \qquad [x](uv) = S([x]u)([x]v).$$

The definition applies to variables and instantiated combinator leaves; abstractions are compiled from the inside out. It has no eta optimization $[x](tx) = t$. Each step preserves the simple type and its expansion is beta-convertible to the corresponding annotated abstraction. This gives an actual SK compiler for the simply typed fragment, not a compiler for arbitrary dependent products or annotations.

For a carrier $X = U_i$, let $E = X \to X$ and $\mathsf{Ch} = E \to E$. A convenient linear family, with all abbreviations expanded when counting, is

$$h = S(KS)(S(KK)\mathsf{id}), \qquad \mathsf{succ} = Sh, \qquad \overline{0} = K\mathsf{id}, \qquad \overline{n+1} = \mathsf{succ}\,\overline{n},$$
$$\overline{1} = Sh(K\mathsf{id}), \qquad \overline{2} = Sh(Sh(K\mathsf{id})).$$

Here $hf \longrightarrow^* S(Kf)$, so $\overline{n}fx \longrightarrow^* f^n x$. The code has $10n + 4$ atoms: h has nine, succ ten, and $\overline{0}$ four. Type payloads are not included in this count; their shapes are fixed for this family. Addition and multiplication can be specified without long expanded trees:

$$\mathsf{add} = [m][n][f][x]\, mf(nfx), \qquad \mathsf{mul} = [m][n][f]\, m(nf).$$

These are finite SK expressions by the displayed algorithm and implement $m + n$ and mn. The numeral type has rank 2 and the operation types rank 3.

Theorem 2.1 (Typed SK compilation)**.** *The intrinsic simply typed compiler preserves original PTS typing, and its expanded output reduces forward to the literal annotated source term. It preserves and reflects native beta conversion of source expansions. For normal source expansions this is equivalent to their literal equality. The numeral counts, ranks and arithmetic beta laws above are checked, including reduction of an unapplied numeral to* $\lambda f : E.\lambda x : X.f^n x$.

This interface does not replace the dependent Hilbert catalogue below.

3 Why a fixed finite typed basis is impossible

Write $\rho(t) = s$ when $t : T$ and $T : U_s$: this is the rank of the *type* of t. In a well-formed context it is unique, invariant under beta conversion between typable terms, and satisfies

$$\rho(fu) \le \rho(f).$$

Every universe atom has type rank zero, since $U_i : U_0 : U_0$. For finitely many fixed closed typed atoms $b_1, \ldots, b_m$, set $R = \max(0, \rho(b_1), \ldots, \rho(b_m))$. Induction on application trees bounds every typable expansion by R, even when every U_i is available. The bound also holds for typable beta-convertible outputs. But define

$$A_0 = U_0, \quad A_{s+1} = \Pi x : A_s.U_0,$$
$$v_0 = U_0, \quad v_{s+1} = \lambda x : A_s.U_0.$$

Then $A_s : U_s$ and $v_s : A_s$, so v_{R+1} cannot be represented.

Theorem 3.1 (No finite fixed typed basis)**.** *No finite list of closed originally typed PTS-rk terms is applicatively complete up to original beta conversion, even with all universe atoms available. The representing application tree must itself be originally typable.*

This excludes a fixed typed SK basis or a single fixed typed combinator, but not numerically indexed families with unbounded type ranks. Such families supply the economical presentation below.

4 Dependent logical schemes

These native logical axioms belong to the independent Hilbert backend. Its identity scheme I_a is not an additional SK surface symbol. Uniform compilation of the entire catalogue to dependent SK remains an obligation; deleting a catalogue family without that proof would change the Hilbert system.

Only the natural numbers in the subscripts are external indices. Every displayed type or family parameter is an ordinary term argument. Define

$$\begin{aligned} I_a &= \lambda A : U_a.\lambda x : A.x, \\ K_{a,b} &= \lambda A : U_a.\lambda B : U_b.\lambda x : A.\lambda y : B.x, \\ P_{a,b} &= \lambda A : U_a.\lambda B : (\Pi x : A.U_b).\Pi x : A.Bx. \end{aligned}$$

Their axiom types are

$$\begin{aligned} I_a &: \Pi A : U_a.\, A \to A, \\ K_{a,b} &: \Pi A : U_a.\Pi B : U_b.\, A \to B \to A, \\ P_{a,b} &: \Pi A : U_a.\Pi B : (\Pi x : A.U_b).\, U_{r(a,b)}. \end{aligned}$$

For dependent substitution, abbreviate the parameter telescope by

$$\Theta_{a,b,c} = \big(A : U_a,\ B : \Pi x : A.U_b,\ C : \Pi x : A.\Pi y : Bx.U_c\big).$$

Here $\lambda\Theta$ and $\Pi\Theta$ bind its three entries in order. The fourth scheme is

$$\begin{aligned} S_{a,b,c} &= \lambda\Theta_{a,b,c}.\lambda f : (\Pi x : A.\Pi y : Bx.Cxy).\lambda g : (\Pi x : A.Bx).\lambda x : A.fx(gx), \\ S_{a,b,c} &: \Pi\Theta_{a,b,c}.\Pi f : (\Pi x : A.\Pi y : Bx.Cxy).\Pi g : (\Pi x : A.Bx).\Pi x : A.Cx(gx). \end{aligned}$$

Logically, I proves reflexive implication, K adds an unused hypothesis, and S combines two deductions using the same hypothesis. The family C allows the conclusion to depend on both that hypothesis and the intermediate proof gx. The combinator P forms the quantified proposition itself.

Theorem 4.1 (Sound core schemes)**.** *For every numerical index, the four displayed terms have exactly the displayed original PTS-rk types. Their fully applied beta laws are*

$$\begin{aligned} I_a\, A\, x &\longrightarrow_\beta^* x, & K_{a,b}\, A\, B\, x\, y &\longrightarrow_\beta^* x, \\ S_{a,b,c}\, A\, B\, C\, f\, g\, x &\longrightarrow_\beta^* fx(gx), & P_{a,b}\, A\, B &\longrightarrow_\beta^* \Pi x : A.Bx. \end{aligned}$$

The ranks of their types are respectively $a+1$, $\max(a+1, b+1)$, $\max(a+2, b+2, c+1)$ *and* $a+2$.

The rank of the type of $P_{a,b}$ is independent of b because its result is a universe symbol, itself of type U_0.

5 A complete Hilbert presentation

Take the sorts $\{U_i : i \in \mathbb{N}\}$, the axioms $U_i : U_0$, and the four preceding schemes. Complete the scheme set $\mathcal{B}$ by the following operations of Bunder–Dekkers, Definition 7.3 [1]. They operate on schemes with sort parameters, before numerical instantiation.

1. **Types of schemes.** From $M : T$ in $\mathcal{B}$, with T not a sort, include $T' : U_s$ whenever the original calculus types this scheme. Here T' is T, or is obtained by replacing a repeated occurrence of a sort parameter in T by a fresh sort parameter.
2. **Expose a product.** Whenever $\mathcal{B}$ contains

$$(\lambda x_1 : A_1 \cdots \lambda x_{k-1} : A_{k-1}.\Pi x_k : A_k.B) : (\Pi x_1 : A_1 \cdots \Pi x_{k-1} : A_{k-1}.U_s),$$

   also include the corresponding well-typed scheme

$$(\lambda x_1 : A_1 \cdots \lambda x_k : A_k.B) : (\Pi x_1 : A_1 \cdots \Pi x_k : A_k.U_t).$$

Only the sort constraints needed for each new scheme's own typing are retained. They are constraints on fixed scheme shapes, not assumptions that the theorem being translated is already derivable. The resulting catalogue has 11 product, 7 identity, 17 constant and 30 substitution shapes [1, Section 7]. Specialize their sort constraints using $U_i : U_0$ and $r(a, b) = \max(a + 1, b)$. Thus one family indexed by a tag from 65 shapes and a finite tuple of natural numbers suffices. The number 65 is an upper bound from this construction, not a minimality result.

The judgment $\vdash_H M : A$ has empty context. Its leaves are the universe axioms and instances of $\mathcal{B}$; its inference rules are

$$\frac{\vdash_H M : \Pi x : A.B \quad \vdash_H N : A}{\vdash_H MN : B[N/x]} \qquad \frac{\vdash_H M : A \quad \vdash_H B : U_s \quad A =_\beta B}{\vdash_H M : B},$$

$$\frac{\vdash_H M : A \quad M \longrightarrow_\beta N}{\vdash_H N : A} \qquad \frac{\vdash_H M : A \quad A \longrightarrow_\beta B}{\vdash_H M : B}.$$

These are application, conversion, subject reduction and type reduction. There is no abstraction or product-formation inference rule: their effect is recovered from the schemes. Annotated lambda and product syntax remains inside the schemes, types and reductions.

PTS-rk is *supersorted*: every constant has a sort and every pair of sorts has a product rule. On well-formed contexts, type correctness and supersortedness identify the repository's sorted-domain abstraction rule with the standard sorted-product rule [1, Theorem 7.2]. Hence Bunder–Dekkers, Theorem 9.4, gives

$$\boxed{\vdash_H M : A \iff \vdash_{\mathrm{rk}} M : A}.$$

This is an external completeness theorem, not a Rocq theorem or axiom of this development. It supplies a Hilbert logical system: closed proofs of exactly the original closed judgments arise from fixed schematic logical axioms and the four rules above. Only its annotated, beta-only presentation is used here.

Theorem 5.1 (Hilbert soundness and proof extraction)**.** *For any catalogue whose original atom typings have been independently proved, a Hilbert derivation of $M : A$ yields a binder-free applicative code c such that*

$$\vdash_{\mathrm{rk}} M : A, \qquad \vdash_{\mathrm{rk}} E(c) : A, \qquad E(c) \longrightarrow_\beta^* M.$$

Here E expands catalogue atoms and preserves application.

The atom-typing premise belongs to the metatheorem, not the definition of a Hilbert derivation. Extraction retains a code through both reduction rules; its expansion recovers the subject by forward beta reduction, not merely conversion. For the literal 65-shape catalogue below, atom typing is discharged: soundness and extraction hold without an additional premise. The extracted code represents a proof through the original typing of $E(c)$; types still use original annotated syntax.

5.1 The literal catalogue and its completeness boundary

A literal catalogue can be described by the four core schemes, their types, and finite prefixes of their parameter telescopes. The types of the core schemes and their successive product exposures are included. For each prefix of length k listed below, put an arbitrary universe U_d at its end and replace the first e products by lambdas, for every $0 \le e \le k$:

Telescope	Prefix lengths	Universe-tail templates, in exposure order
P	$2; 3$	P_2, P_3, P_8; P_7, P_9, P_{10}, P_{11}
I	2	I_5, I_6, I_7
K	$2; 3; 4$	K_{12}–K_{14}; K_{15}–K_{18}; K_7–K_{11}
S	$3; 4; 5; 6$	S_9–S_{12}; S_{13}–S_{17}; S_{18}–S_{23}; S_{24}–S_{30}

Length zero gives a universe; length one gives $P_5 = \Pi A : U_a.U_d$ and $P_4 = \lambda A : U_a.U_d$. The S prefix of length two is the P prefix of length two. The length-three P telescope is $(A : U_a, B : \Pi x : A.U_b, x : A)$; its terminal body Bx supplies $P_6 = \lambda A : U_a.\lambda B : (\Pi x : A.U_b).\lambda x : A.Bx$. The other telescopes are the full binder lists of I, K, S above. All domain-sort slots and the final slot d are independent. The labels are local names, not an identification with the paper's numbering.

These templates give 11 product, 7 identity, 18 constant and 30 substitution shapes. Every instance has a proved original typing and exact type rank. Every declared type is literally another catalogue subject or a universe; its Hilbert type judgment and every leading-product exposure are derivable. The redundant constant shape $K_{13}(a, b, c) = \lambda A : U_a.\Pi B : U_b.U_c$ has the independently typed representation

$$K_{0,0}\, U_1\, U_a\, (P_5(b, c)) \longrightarrow_\beta^* \lambda A : U_a.\Pi B : U_b.U_c.$$

Removing it leaves 65 shapes and exactly the same Hilbert judgments. The remaining P_4, P_5 also have independently typed representations in the four-family core, with forward beta recovery.

Corollary 5.2 (Completeness of the literal catalogue)**.** *The 65-shape Hilbert system derives exactly the closed judgments of original PTS-rk. This mathematical result uses [1]; its completeness direction is not yet a Rocq theorem.*

To prove it, first retain all 66 templates. A declared type is a universe, a core type, or a pure-product prefix ending in a universe. Freshening a repeated sort occurrence in operation

(1) therefore only renames an independent domain slot or frees the final slot; the table contains every well-typed instance. Operation (2) increments e, and all exposures are present. The original functional sort rules fix each declared type. Thus the 66 templates contain the schematic catalogue of Definition 7.3; Theorem 9.4 supplies completeness by inclusion. Soundness supplies the reverse implication, and the checked 65/66 equivalence removes K_{13}. The symbolic template-inclusion step is not formalized in Rocq.

The formalized support includes contextual start and empty-context equivalence, internal substitution, classification of the seven variable/application-ended heads, literal product-exposure compatibility and beta simulation, I, K, S abstraction and product assembly given the required family derivations, and the local type-preserving six-to-three S contraction with shared premises. General chain shortening with a sharing-aware measure, internal type correctness and unconditional deduction admissibility remain unformalized. The badges certify the supporting results, not the completeness direction.

5.2 A Metamath Zero specification

This Hilbert calculus has a first-order specification in Metamath Zero (MM0) [6]. The accompanying `CombinatoryPTS-rk.mm0` uses strict sorts for natural-number indices, raw de Bruijn terms, finite ranked telescopes and provable statements. Lambda annotations remain part of the term syntax; beta reduction retains capture-avoiding substitution. Auxiliary Horn rules compute ranks, shifts, substitutions and reductions. The graph `form e L D E M T` folds a ranked telescope `L` over a body/type pair `D:E`, exposing `e` initial lambdas and leaving products afterwards. Only fixed axiom rows feed Hilbert typing: they generate the 11 product, 7 identity, 17 constant and 30 substitution shapes, without an opaque catalogue lookup or original-typing oracle.

Its Hilbert inference layer is the following literal source excerpt:

```
term h: tm > tm > judgment;
axiom h_universe (i: nat): $ h (u i) (u z) $;
axiom h_scheme (M A: tm): $ ax M A $ > $ h M A $;
axiom h_apply (f x A B T: tm):
  $ h f (pi A B) $ > $ h x A $ > $ sub z x B T $ > $ h (app f x) T $;
axiom h_convert (M A B C: tm) (r: nat):
  $ h M A $ > $ h B (u r) $ > $ red A C $ > $ red B C $ > $ h M B $;
axiom h_subject (M N A: tm): $ h M A $ > $ red M N $ > $ h N A $;
axiom h_type (M A B: tm): $ h M A $ > $ red A B $ > $ h M B $;
```

Here an MM0 `axiom` declares a rule of this specified logical system; the arrow between dollar-delimited statements separates premises from conclusion. It is not a new assumption in the Rocq development. Read the rules by their least syntactic derivability relation, not as a claim that every first-order model has only the intended numbers or terms. Only the fixed closed templates and universe terms initialize Hilbert typing; application, conversion and reduction then preserve closedness. In `h_convert`, `C` is a common beta reduct of the two types; original confluence identifies this condition with beta conversion. An MM0 proof certifies such a derivation, while the PTS subject is the proof object it types. The specification and ten example derivations[2], including capture-avoiding substitution, pass the MM0 elaborator and reference checker. These checks do not establish the general Rocq correspondence or source completeness. The full source is included in Appendix A, directly from the same file; it is longer than one page.

[2] `../../docs/direct-r-handoff/verification/combinatory-mm0-2026-10-22/CombinatoryPTS-rk-examples.mm1`

6 A first-order algebraic presentation as QIIRT

A generalized algebraic theory (GAT) specifies dependent sorts, operations and equations; its initial algebra is syntax modulo those equations [3, 4]. Here we present a contextual dependent SK signature with a product former, separately from the full Hilbert catalogue. The natural-model presentation [5] keeps the context as an index and computes each term's type recursively. The result is a quotient inductive-inductive-recursive type (QIIRT): its sorts, paths and typing function are defined together. The following excerpts come directly from the checked Cubical Agda source, Combinatory.agda[3].

```
data Con : Set                -- Contexts of typed assumptions.
data Sub : Con → Con → Set    -- Sub Γ Δ replaces Δ-variables in Γ.
data Ty : Con → Nat → Set     -- Ty Γ r: types A : U_r in Γ.
data Tm : Con → Set           -- Tm Γ: terms; tyOf computes their type.

RankedTy : Con → Set
RankedTy Γ = Σ Nat (Ty Γ)

tyOf : ∀ {Γ} → Tm Γ → RankedTy Γ
```

Here $\mathrm{Ty}(\Gamma, r)$ represents types classified by U_r. Thus $\mathrm{RankedTy}\,\Gamma = \sum_r \mathrm{Ty}(\Gamma, r)$ retains the sort of the type, and the fibre

$$\mathrm{TmAt}(\Gamma, r, A) = \sum_{t:\mathrm{Tm}\,\Gamma} (\mathrm{tyOf}\,t = (r, A))$$

recovers terms at a specified type. Equality constraints serve in place of a type index: this is the Ford transformation, not untyped syntax.

6.1 Operations and equations

The context operations are empty context and extension $\Gamma \triangleright A$. Substitutions have identity, composition, the unique substitution to the empty context, guarded pairing, and projections. Write their actions as $A[\sigma]$ and $t[\sigma]$. In addition to the category laws, impose

$$\begin{gathered} A[\mathrm{id}] = A, \qquad A[\sigma][\nu] = A[\sigma \circ \nu], \\ t[\mathrm{id}] = t, \qquad t[\sigma][\nu] = t[\sigma \circ \nu], \\ \mathrm{fst}(\sigma, t) = \sigma, \quad \mathrm{snd}(\sigma, t) = t, \quad (\mathrm{fst}\,\sigma, \mathrm{snd}\,\sigma) = \sigma, \\ (\sigma, t) \circ \nu = (\sigma \circ \nu, t[\nu]). \end{gathered}$$

Pairing requires $\mathrm{tyOf}\,t = (r, A[\sigma])$. The singleton $\langle t \rangle = (\mathrm{id}, t)$ uses the type-identity path; lifting a substitution through a binder uses the type-composition path. These structural guards are proved in the source. Comprehension eta is a law of substitutions, not function eta.

The logical operations are universe types $U_i : \mathrm{Ty}(\Gamma, 0)$, universe terms $\mathrm{univ}_i : \mathrm{Tm}\,\Gamma$, guarded decoding El_r, dependent products and application. Their types satisfy

$$\begin{gathered} \mathrm{tyOf}(\mathrm{univ}_i) = (0, U_0), \qquad \mathrm{El}_0(\mathrm{univ}_i) = U_i, \\ \frac{A : \mathrm{Ty}(\Gamma, a) \quad B : \mathrm{Ty}(\Gamma \triangleright A, b)}{\Pi A B : \mathrm{Ty}(\Gamma, \max(a+1, b))}. \end{gathered}$$

[3] `agda/Combinatory.agda`

A codomain is syntax in an extended context, not a meta-function on terms. Application is exported with the usual argument-type guard:

```
app : ∀ {Γ a b} (A : Ty Γ a) (B : Ty (ext Γ A) b) (f x : Tm Γ)
    → tyOf f ≡ (max (suc a) b , Pi A B)
    → tyOf x ≡ (a , A) → Tm Γ
app {a = a} A B f x pf px = app-guarded A B f x pf
  (px • λ i → a , sym (ty-id A) i)
```

The primitive guard uses $A[\mathrm{id}]$ to keep recursive definitions strictly positive; the displayed wrapper derives it from the identity law. Substitution is natural for universes, decoding and products, with $(\Pi AB)[\sigma] = \Pi(A[\sigma])(B[\sigma \uparrow_A])$. The closed atoms are $S_{a,b,c}, K_{a,b}$ and $P_{a,b}$, with the dependent telescopes of Section 4. Typed application spines fix the saturation arities of the guarded S, K laws. Identity is not an additional atom. The derived `skk-computes` law composes the S and K laws for certified instantiation spines; constructing every needed closed polymorphic instantiation remains a coverage obligation. The P law is $\mathrm{El}_{r(a,b)}(P_{a,b}AB) = \Pi x : \mathrm{El}_a(A).\mathrm{El}_b(Bx)$, with the displayed applications and decodings well typed. There is no abstraction constructor, function eta or unrestricted quotation $\mathrm{Ty}(\Gamma, r) \to \mathrm{TmAt}(\Gamma, 0, U_r)$.

For a term path $L = R$, recursion must also supply a path between its types. The guarded path constructors therefore carry $e : \mathrm{tyOf}\, L = \mathrm{tyOf}\, R$, with recursive clause $\mathrm{tyOf}(\mathrm{law}\ e\ i) = e(i)$. Only the specified laws generate paths, not arbitrary pairs of equally typed terms. All four sorts are sets: the term-truncation coherence guard is derived from the sethood of $\mathrm{RankedTy}\,\Gamma$. Structural contraction already follows from comprehension: in $\Gamma \triangleright A$, the substitution (id, q_A) maps into $\Gamma \triangleright A \triangleright A[p]$. Substituting along it shares one variable. This is distinct from compiling every dependent term into closed S, K atoms; ordinary contextual application itself permits shared variables.

This gives executable, intrinsically typed quotient syntax. The source derives the structural guards; general admissibility of the logical equations' coherence guards, interpretation in original annotated PTS-rk, equality reflection and dependent combinatory completeness remain separate obligations.

6.2 Binder-free normalization and bidirectional checking

A direct binder-free normalizer is possible without expanding combinators to lambda terms. The checked core BinderFreeCore.agda[4] uses simple types $A ::= \iota \mid A \to A$, explicitly instantiated S, K, application and inert ground labels. This is an independent test calculus, not an extension of PTS-rk by ground constants. Its normal forms are unsaturated heads with normal arguments: S, Sf, Sfg, K, Kx, and ground labels. Their typing is intrinsic. Since types do not depend on terms, ordinary indexed induction suffices; induction–recursion adds no benefit here.

Normalizing application contracts a head when its last argument arrives. Termination is proved by a computability family, recursive on simple types:

$$\begin{aligned} \mathsf{Good}_\iota(n) &= 1, \\ \mathsf{Good}_{A\to D}(f) &= \prod_{x:\mathsf{Nf}(A)} \big(\mathsf{Good}_A(x) \to \sum_{y:\mathsf{Nf}(D)} (f \bullet x \Downarrow y) \times \mathsf{Good}_D(y)\big). \end{aligned}$$

Here $\Downarrow$ is independently generated by the literal combinator rules. Structural induction

[4] `agda/BinderFreeCore.agda`

on normal forms constructs `good`; it is not an input termination assumption. The same construction yields both application and its forward reduction certificate:

```
record Result {A : Ty} (t : Tm A) : Set where
  constructor result
  field
    normal : Nf A
    sound : t ↠ erase normal
open Result public

apply-normal : (f : Nf (A ⇒ D)) (x : Nf A)
             → Σ (Nf D) (λ y → (f • x ⇓ y) × Good D y)
apply-normal f x = good f x (good x)

normalize-apply : (f : Nf (A ⇒ D)) (x : Nf A)
                → Result (erase f · erase x)
normalize-apply f x with apply-normal f x
... | y , p , py = result y (application-sound p)
```

The full normalizer folds over application trees. Every output is irreducible, every intermediate in its certificate retains the input type, and a ground normal form is a ground label. These are statements about weak combinatory reduction, including reduction in all saved arguments; they do not assert termination of every possible reduction sequence or transfer the PTS rank/size bounds to this encoding. agda (proof)

The bidirectional interface synthesizes each explicitly typed head, checks an application's argument at the synthesized domain, and compares simple types structurally. `check-normalize` decides an independently defined declarative typing judgment and returns its elaborated normal form. Rejection refutes every such typing derivation; successful checking cannot be followed by normalization failure. No type-inference oracle or fuel is supplied. Inductive equality witnesses implement positive casts so that the executable examples compute even in Cubical Agda. agda (proof)

This also integrates with a genuine quotient. Let $\equiv_w$ be the equivalence generated by the two combinator rules in application contexts. Normalization respects every rule and reflects conversion:

$$t \equiv_w u \quad \Longleftrightarrow \quad \mathsf{nf}(t) = \mathsf{nf}(u).$$

Decidable equality of normal forms therefore decides weak conversion. The set quotient by $\equiv_w$ is isomorphic to the normal-form family:

```
Quotient : Ty → Set
Quotient A = Tm A Q./ WeakConversion

normalizeQ : Quotient A → Nf A
normalizeQ = Q.rec normal-isSet nf (λ _ _ → conversion-coherent)

quoteQ : Nf A → Quotient A
quoteQ n = Q.[ erase n ]

normalize-quote : (n : Nf A) → normalizeQ (quoteQ n) ≡ n
normalize-quote = nf-erase

quote-normalize : (q : Quotient A) → quoteQ (normalizeQ q) ≡ q
quote-normalize = Q.elimProp (λ q → Q.squash/ _ _)
  (λ t → sym (Q.eq/ t (erase (nf t)) (steps-conversion (nf-sound t))))
```

```
quotient-normal-iso : Iso (Quotient A) (Nf A)
quotient-normal-iso = iso normalizeQ quoteQ normalize-quote
↪  quote-normalize
```

The quotient's application uses the same `appNF` as the checker and normalizer, with a proof that it represents ordinary application. Thus the quotient requires no second normalization algorithm. agda (proof) agda (proof) agda (proof)

The exact PTS boundary. This quotient cannot replace the PTS-rk QIIRT: at $A = U_0$, the distinct weak normal forms SKK and $SK(SKK)$, with the appropriate independent type instances, have the same annotated beta-normal expansion $\lambda x : A.x$. Conversely, $S(K(SKK))$ has beta-normal expansion $\lambda g : A \to A.\lambda x : A.gx$, which is not beta-convertible to SKK instantiated at $A \to A$, whose normal expansion is $\lambda g : A \to A.g$. Both claims, including native typing and the latter non-conversion, are checked in BinderFreeBoundary.agda[5]. Thus weak normal-form comparison is too discriminating, while adding function eta would identify too much. Neither issue is fixed by bidirectional typing. agda (proof)

6.3 Canonical beta representatives

Write E for annotated SK expansion and NF_β for the native normalizer. Every intrinsic SK term has an originally typed expansion. The native normalizer computes a sufficient rank from that derivation, so `source-beta?` decides beta equality of expansions without fuel or a termination assumption. Its reference output retains original typing, normality and a bounded forward trace. agda (proof)

Canonical SK outputs require retaining the compiler's source fragment. `Source Γ A` is intrinsic annotated simple lambda syntax, with a proof of original PTS typing after erasure e. Capture-avoiding substitution and the native greedy passes lift to this syntax; for every budget d,

$$e(\mathsf{rounds}_{\mathrm{Source}}(d, s)) = \mathsf{rounds}_{\mathrm{native}}(d, e(s)).$$

The source normalizer n uses a syntax-only budget: variables and ground labels contribute zero, abstraction keeps its body's budget, and application takes the maximum of its children's budgets and its head-redex rank. The last contribution is the simple type order of a literal lambda head, and zero for other heads. Simple-type annotations are already normal. This budget is proved sufficient for the original native rank relation; computing it never evaluates typing proofs. On normal source arguments, the application budget is exactly that head contribution. agda (proof)

Erasure is injective at each fixed context and simple type, including the hidden intermediate types of applications. Thus $e(ns) = \mathrm{NF}_\beta(e(s))$ and beta-equal erasures yield literally equal normalized sources. No parser for arbitrary native terms, or replacement for original typing, is assumed. agda (proof) agda (proof) agda (proof)

Deterministic bracket compilation C satisfies $E(Cs) \longrightarrow_\beta^* e(s)$. Every closed native-normal source compiles to an actual irreducible SK normal form. Combining expansion into `Source`, its normalizer, and compilation gives the checked operation $\kappa(c) = C(n(\mathsf{expandSource}(c)))$: agda (proof) agda (proof)

```
normalize : SK.Tm A → SK.Tm A
normalize t = C.compile (SN.normalize (C.expand-source t))
```

[5] `agda/BinderFreeBoundary.agda`

```
sound : (t : SK.Tm A) → Beta (B.expand {zero} (normalize t)) (B.expand t)
sound t = beta-trans (steps-beta (C.compile-steps (SN.normalize
↪  (C.expand-source t))))
  (beta-trans (beta-sym (steps-beta (SN.reduces (C.expand-source t))))
    (beta-path (C.erase-expand t)))

coherent : (t u : SK.Tm A)
  → Beta (B.expand {zero} t) (B.expand u) → normalize t ≡ normalize u
coherent t u e = cong C.compile (SN.coherent (C.expand-source t)
↪  (C.expand-source u)
  (beta-trans (beta-path (C.erase-expand t))
    (beta-trans e (beta-sym (beta-path (C.erase-expand u))))))

idempotent : (t : SK.Tm A) → normalize (normalize t) ≡ normalize t
idempotent t = coherent (normalize t) t (sound t)

reflects : (t u : SK.Tm A)
  → normalize t ≡ normalize u → Beta (B.expand {zero} t) (B.expand u)
reflects t u e = beta-trans (beta-sym (sound t))
  (beta-trans (beta-path (cong B.expand e)) (sound u))
```

For codes of the same simple type, the resulting laws are

$$E(\kappa c) =_\beta E(c), \qquad \kappa(\kappa c) = \kappa c, \qquad \kappa c = \kappa d \iff E(c) =_\beta E(d).$$

Define the stricter family $\mathsf{Canonical}(A) = \sum_{c:\mathsf{Code}(A)} (\kappa c = c)$. Its members are irreducible SK terms with decidable equality. The actual set quotient of SK codes by native beta equality is isomorphic to this family, and the isomorphism respects application:

```
apply-and-normalize : Canonical (A SK.⇒ D) → Canonical A → Canonical D
apply-and-normalize f x = canonical (fst f SK.· fst x)

apply-sound : (f : Canonical (A SK.⇒ D)) (x : Canonical A)
  → Beta (B.expand {zero} (fst (apply-and-normalize f x)))
      (B.expand (fst f SK.· fst x))
apply-sound f x = sound (fst f SK.· fst x)
```

`check-canonical` decides the independent declarative SK typing judgment and returns a beta-faithful canonical elaboration. Identity, both identity spines, identity application and canonical application compute by `refl` in the checked examples; the eta pair remains provably distinct. agda (proof) agda (proof)

Scope. Outputs are binder-free, but this reference algorithm uses typed lambda syntax internally. Its soundness is native beta equivalence, not a forward weak-SK trace from the input. Full PTS integration still requires dependent catalogue coverage, compatibility with all QIIRT paths, and soundness/completeness against the original six typing rules; bidirectional checking alone supplies neither. Requiring $E(c)$ itself to be normal is too restrictive because SK expansions have administrative beta redexes. Eta expansion is not permitted. The algebraic equivalence of [12] concerns the extensional simply typed/untyped case; it is not that missing beta-only dependent theorem. The compact core shares its normalizer, but covers a smaller theory: a reduction in the code needed for the full PTS presentation is not yet established.

6.4 Direct evaluation and its canonical-image boundary

Every weak SK step expands to a forward original annotated beta trace. Consequently direct weak normalization can precede κ without changing its result, and the map from the weak

quotient to the native-beta quotient preserves application. This map need not be injective. agda (proof)

At **ground output type** ι, every weak normal form is a ground label, which κ fixes. Thus direct normalization already equals κ for every ground-typed input. Weak conversion and native beta conversion coincide there, giving a direct conversion decision and the following canonical application, whose computational projection constructs no lambda intermediate:

```
normalize : SK.Tm SK.ι → SK.Tm SK.ι
normalize = Direct.direct

agrees : (t : SK.Tm SK.ι) → normalize t ≡ Canonical.normalize t
agrees t = sym (Direct.canonical-after-direct t •
↪  ground-normal-canonical (Weak.nf t))

canonical : SK.Tm SK.ι → Canonical.Canonical SK.ι
canonical t = normalize t , ground-normal-canonical (Weak.nf t)

apply-and-normalize : Canonical.Canonical (A SK.⇒ SK.ι)
  → Canonical.Canonical A → Canonical.Canonical SK.ι
apply-and-normalize f x = canonical (fst f SK.· fst x)

application-agrees : (f : Canonical.Canonical (A SK.⇒ SK.ι))
  (x : Canonical.Canonical A)
  → apply-and-normalize f x ≡ Canonical.apply-and-normalize f x
application-agrees f x = Canonical.canonical-path (agrees (fst f SK.·
↪  fst x))
```

agda (proof)

Function-valued results require more. With independent type instances understood, the canonical inputs $F = S(KK)(SKK)$ and $a = SKK$ represent $\lambda f : \iota \to \iota.\lambda x : \iota.f$ and $\lambda y : \iota.y$. Direct weak application yields $K(SKK)$, with four atoms, whereas $\kappa(Fa) = S(S(KS)(KK))(KK)$ has eight. Their inequality and failure of canonical-image closure are proved, not just evaluated. This rules out using weak application alone, not a stronger direct evaluator. An abstraction-sensitive closure/spine evaluator must preserve arrow-typed neutrals without eta expansion and prove agreement with κ. agda (proof)

6.5 A normal-form-first dependent algebra

Normal forms can replace syntax modulo conversion: store canonical types and terms, and make application and substitution **operations returning normal forms**, not unrestricted constructors of that family. In the simply typed fragment this needs only an ordinary dependent pair:

```
Term : Set
Term = Σ SK.Ty C.Canonical

tyOf : Term → SK.Ty
tyOf = fst

TmAt : SK.Ty → Set
TmAt A = Σ Term (λ t → tyOf t ≡ A)

embed : C.Canonical A → TmAt A
embed {A} t = (A , t) , refl

at : TmAt A → C.Canonical A
at ((B , t) , p) = subst C.Canonical p t

app : TmAt (A SK.⇒ D) → TmAt A → TmAt D
app f x = embed (C.apply-and-normalize (at f) (at x))
```

The computed-type fibre is proved isomorphic to `Canonical A`; application has its result type by computation and agrees with canonical application. No beta-quotient path constructor is needed in this carrier. This is not a dependent IIRT: simple types are independent of terms. agda (proof)

There is also a genuinely dependent, checked ingredient. Over original contexts, substitutions can store normal images and mere original typing evidence:

```
record SubNF (Γ : Context m) (Δ : Context n) : Set where
  constructor sub
  field
    sub-images : Fin m → Term n
    sub-normal : ∀ i → Normal (sub-images i)
    sub-typed : ∥ TypedSub Γ Δ sub-images ∥₁
```

Normalize every image after composition. Reducing those images also reduces each substituted context lookup; original subject reduction supplies the sorting premise required to convert the image's dependent type. Normalization coherence makes the result independent of the truncated typing witnesses. The resulting set has proved left/right identity and associativity paths. These are actual carrier equalities, not equality constructors imposed by a quotient. Conversion of substitutions is exactly equality of their normalized images. Only typing witnesses are truncated. agda (proof)

The dependent families use the original six-rule judgment directly:

```
record NfAt (Γ : Context n) (A : Term n) : Set where
  constructor normal-at
  field
    term : Term n
    normal : Normal term
    typed : ∥ Typ Γ term A ∥₁
open NfAt public

TyNF : Context n → Nat → Set
TyNF Γ r = NfAt Γ (U r)
```

For a normal type A, its inhabitants are `NfAt` at the stored code of A. Normalize both the substituted subject and its type. This defines $A[\sigma]$ and $t[\sigma]$ on these families, with proved identity and composition paths; term equations include transport along the corresponding type equation. The smart cast `castNF` transports only the typing certificate, leaving the stored syntax definitionally unchanged. It is proved equal to ordinary transport of the entire record.

Thus proof transport need not enter the normalizer's computational input. agda (proof)

Context comprehension has terminal substitutions, projections, pairing and last-variable operations. Their projection, uniqueness and context-eta laws are literal carrier paths. Lifting targets the extension by the **normalized** pulled-back declaration; original context conversion proves its typing. Pairing is natural under composition, and the projection–lifting square commutes. Context eta here is not function eta. agda (proof)

Products and abstractions of normal components are normal structurally. Application computes a normal dependent result type and a normal inhabitant:

```
appNF : {Γ : Context n} (wf : WellFormed Γ) (A : TyNF Γ a)
  (B : TyNF (extend Γ (term A)) b)
  → NfAt Γ (term (piNF A B)) → (x : NfAt Γ (term A))
  → NfAt Γ (term (appType wf A B x))
appNF wf A B f x = convertNF (appType wf A B x) (appType-sound wf A B x)
  (normalizeProof {t = app (term f) (term x)} {A = substitute (term B)
  ↪  (term x)}
    wf (PT.rec2 squash₁
    (λ hf hx → ∣ application hf hx ∣₁) (typed f) (typed x)))
```

The result's original typing and normality are fields, not assumed closure laws. Annotated beta, product/abstraction/application naturality and universe substitution are proved. Decoding a normal inhabitant of U_r is literally the same family as `TyNF` at rank r; no separate decoding quotient is needed. agda (proof)

The computed-type interface stores a normal subject and mere original typability, with no chosen type. Type uniqueness determines its normal `tyOf` and rank independently of all witnesses. Its fibre at each normal type is isomorphic to `NfAt`; every original derivation normalizes into this interface, with forward beta reduction and conversion preservation/reflection. agda (proof)

Normal declarations also support an actual inductive-recursive context presentation:

```
mutual
  data Con : Set where
    ε : Con
    ext : (Γ : Con) {r : Nat} → TyNF (context Γ) r → Con

  length : Con → Nat
  length ε = zero
  length (ext Γ A) = suc (length Γ)

  context : (Γ : Con) → Context (length Γ)
  context ε = empty
  context (ext Γ A) = extend (context Γ) (term A)

Ty : Con → Nat → Set
Ty Γ r = TyNF (context Γ) r

Tm : (Γ : Con) {r : Nat} → Ty Γ r → Set
Tm Γ A = NfAt (context Γ) (term A)

Sub : Con → Con → Set
Sub Γ Δ = SubNF (context Γ) (context Δ)
```

The mutual context/decoding definition is checked with positivity and termination enabled. Every such context is merely well formed; coherence allows normalization and type/term action without choosing a well-formedness witness from that truncation. agda (proof)

The dependent tests include the neutral type $f\,U_0$ under $\sigma(f) = \lambda X : U_0.U_0$: its pulled-back type computes to U_0. The polymorphic identity $\lambda A : U_0.\lambda x : A.x$, its specialization at U_0, and subsequent application to U_7 test dependent type changes, not only simply typed application. agda (proof)

6.5.1 Raw typing and the remaining presentation boundary

For arbitrary original annotated input, structural synthesis returns either a normalized inferred type with original typing evidence, or a refutation of every original typing derivation:

```
record Inferred (Γ : Context n) (t : Term n) : Set where
  constructor inferred
  field
    inferred-type : Term n
    type-normal : Normal inferred-type
    type-rank : Nat
    type-sorted : Typ Γ inferred-type (U type-rank)
    subject-typed : Typ Γ t inferred-type
open Inferred public

data Result (Γ : Context n) (t : Term n) : Set where
  accepts : Inferred Γ t → Result Γ t
  rejects : (∀ {A} → Typ Γ t A → ⊥) → Result Γ t
```

`infer` recurses on raw syntax, including annotations. It normalizes a type only after proving that type sorted. Application inspects the inferred normal function type and compares the normal argument type with its domain. Original inversion, type uniqueness and confluence prove all rejection branches complete. Consequently `check wf t A` decides the original $\Gamma \vdash t : A$ for well-formed Γ, with no certificate input or replacement typing relation. `check-context` also decides context validity; on an arbitrary raw context it decides the conjunction of well-formedness and typing, not bare typing in a malformed context. agda (proof)

This establishes the normal-form-first algebra for dependent operations and the original raw typing decision. It does **not** yet replace the combinatory QIIRT: the normalizer still uses annotated native binders. Its algebra laws must be assembled into an interpretation respecting every QIIRT path and the dependent $S/K/P$ families, with source completeness, context adequacy and sethood of the inductive-recursive context carrier. The checked mutual definition above concerns contexts and decoding, not a new simultaneous definition of all normal syntax and hereditary evaluation. The latter still needs a positive, terminating definition covering redexes inside annotations. No reduction in the total full-presentation code is claimed.

Normal-form-first removes conversion-quotient elimination and repeated guard handling; it does not remove coherence, which becomes laws of normalizing operations. These laws need not be judgmental on open inputs, and no function eta is introduced. The QIIRT paper [5, Sections 3.2, 4.3–4.4 and 5] similarly distinguishes deferred guards from proved laws and strictification of an existing model from defining initial syntax; its normalization example is only the substitution calculus.

7 Elementary normalization and finite syntax carriers

7.1 The rank parameter and the reduction strategy

Let $|t|$ count all constructors of the original annotated syntax, including annotations. Use Knuth's tetration notation $a \uparrow\uparrow 0 = 1$ and $a \uparrow\uparrow (h+1) = a^{a\uparrow\uparrow h}$, and put $q = \max(2, N)$. The

relevant bound d bounds the type rank of **every redex head** in t, also inside annotations. It is the repository's `RedexBound`, not merely the outer type rank $\rho(t)$. Indeed the closed terms

$$t_a = (\lambda x : (\Pi A : U_a.A \to A).U_0)I_a$$

all have type U_0 and 13 nodes, while their root redex has rank $a + 2$. This separates the two rank notions; it is not a lower bound on the cost of reducing these particular terms.

Theorem 7.1 (Normal form and intermediate-size bound)**.** *Suppose* Γ *is well formed,* $\Gamma \vdash t : A$*,* $|t| = N$*, and* `RedexBound` *holds at* d*. There is a deterministic beta-reduction sequence to a normal* t^* *with* $\Gamma \vdash t^* : A$*. Every intermediate term, including* t^**, has at most* $q \uparrow\uparrow (d + 1)$ *nodes and retains redex-rank bound* d*. The documented abstract normalization cost is at most* $(N + 5d + 2) \uparrow\uparrow (2d + 2)$*.*

Use d passes of the existing greedy normalizer [8]. Within a pass, reduce the annotation or left child first, then the body or right child, then contract the root when possible. A pass lowers the maximal redex rank; sequentialization bounds every single-beta intermediate by $2^{|t|}$. Iterating this binary exponential d times gives a bound no larger than $q \uparrow\uparrow (d + 1)$; the comparison is checked. Original subject reduction and redex-rank preservation supply the typing and rank invariants. For fixed d, these functions are elementary, with tower heights linear in d.

For any fixed finite set of atom shapes, including the indexed catalogue, expansion satisfies $|E(c)| \le C\ell$ when c has ℓ atoms, with a shape-dependent constant C. Thus $\max(2, C\ell) \uparrow\uparrow (d+1)$ bounds normalization of its expansion. For type-indexed atoms the bound requires fixed type shapes, as in the numeral family; arbitrary type-index payloads must also be charged. For the simply typed compiler, let $a(c)$ count SK/ground leaves, counting each variable as one atom on open codes, and let $s(t)$ count source constructors. Bracket abstraction satisfies $1 + a([x]c) \le 4a(c)$; consequently $a(Ct) \le 4^{s(t)}$. Source node count is at most annotated native node count. For $N = |E(c)|$ and $d = \mathsf{budget}(\mathsf{expandSource}(c))$, normalization and compilation therefore give the output bound

$$a(\kappa(c)) \le 4^{\max(2,N)\uparrow\uparrow(d+1)}.$$

The source proves the sharper executable bound `4^(tower d N)`; the displayed form uses the tower comparison above. This is elementary at fixed d, with one additional exponential. It bounds output **atoms**, not their type-index payloads, all compilation intermediates, or runtime memory. The dependent compiler remains open. agda (proof) Numerical labels count as one node here; their bit lengths must be charged in a machine-level complexity claim.

7.2 Finite normal families and dependent application

Rank and length alone do not make normal forms finite: the infinitely many distinct $U_i : U_0$ are all closed normal terms of size one and type rank zero. Fix instead a finite alphabet Σ of universe labels. For a fixed type A, the desired family of **syntax codes** is

$$\mathrm{NF}_\Sigma(\ell, r; A) = \{t \mid \mathrm{labels}(t) \subseteq \Sigma, |t| = \ell,\ t \text{ normal},\ \vdash t : A,\ \vdash A : U_r\}.$$

This is a proposition-valued fibre of finite syntax, not a collection counting arbitrary typing derivations. The checked Agda construction below uses genuine normal representatives and an injective finite encoding. Universe labels and de Bruijn variables range over finite alphabets; products and lambdas retain both their annotation and body.

Theorem 7.2 (Finite carriers)**.** *Beta reduction introduces no universe labels. All closed terms of at most s nodes over Σ, and hence all their normal subfamilies, have an explicit complete finite enumeration of length at most $(3s+|\Sigma|+4) \uparrow\uparrow 3$ (duplicates are harmless). Consequently, for a closed typed d-bounded input of size N, every state of the preceding strategy lies in a finite carrier of size at most*

$$(4N+4) \uparrow\uparrow (d+3), \qquad \Sigma = \mathrm{labels}(t).$$

For closed normal $f : \Pi x : A.B$ and $y : A$, with $\Pi x : A.B : U_r$, let $N = 1 + |f| + |y|$. Running r passes on fy returns a normal $z : B[y/x]$. Every intermediate has size at most $\max(2, N) \uparrow\uparrow (r+1)$ and lies in the finite raw carrier over $\mathrm{labels}(f) \cup \mathrm{labels}(y)$ of capacity at most $(4N+4) \uparrow\uparrow (r+3)$.

The last rank bound needs no internal rank inference: normal f, y have no internal redexes, so only the new root redex matters. Its rank is r. The algorithm can represent each term state by an index into the displayed finite list; the proofs check the underlying normalizer and its finite envelope, not an extracted finite-memory machine. They do not bound the storage of arbitrary typing witnesses. Nor is an externally supplied, possibly beta-expanded B bounded by $|f| + |y|$: count supplied type syntax and its rank budget separately if types themselves are to be normalized.

7.3 Inductive-inductive-recursive normal representatives

The quotient syntax of Section 6 identifies beta-equivalent terms of different lengths. Length must therefore be measured on explicit representatives. NormalForms.agda[6] defines normal syntax, a non-lambda-head family, and their raw interpretation together:

```
module _ (Label : Set) where
  data Nf (n : Nat) : Set
  data NonLamNf {n : Nat} : Nf n → Set
  erase : ∀ {n} → Nf n → Raw Label n

  data Nf n where
    nf-U   : Label → Nf n
    nf-var : Fin n → Nf n
    nf-pi nf-lam : Nf n → Nf (suc n) → Nf n
    nf-app : (f x : Nf n) → NonLamNf f → Nf n

  data NonLamNf {n} where
    non-lam : ∀ {f : Nf n} → NotLambda (erase f) → NonLamNf f

  erase (nf-U i) = U i
  erase (nf-var i) = var i
  erase (nf-pi A B) = pi (erase A) (erase B)
  erase (nf-lam A b) = lam (erase A) (erase b)
  erase (nf-app f x p) = app (erase f) (erase x)
```

The occurrence of `erase` in the `non-lam` constructor makes this a genuine inductive-inductive-recursive definition. Here `n` is context length, while `erase` retains every original annotation; it forgets only the normal-form presentation. `NotLambda` excludes precisely lambda heads. Thus application cannot introduce a beta redex, and all children, including annotations, are normal. Conversely, every normal raw term has such a representative. The head witnesses are propositions and `erase` is injective. agda (proof)

[6] `agda/NormalForms.agda`

Exact and bounded node counts give the fibres

```
SizedNf : (a n s : Nat) → Set
SizedNf a n s = Σ (Nf (Fin a) n) (λ t → nodes (erase (Fin a) t) ≤ s)

ExactNf : (a n N : Nat) → Set
ExactNf a n N = Σ (Nf (Fin a) n) (λ t → nodes (erase (Fin a) t) ≡ N)
```

An `AtMost A k` certificate is a function `A → Fin k` together with its injectivity proof. Depth-bounded raw syntax has a sum/product encoding equivalent to `Fin (capacity a n s)`; every node-bounded term encodes and decodes identically. The arithmetic proof bounds this capacity by the displayed tetration. No normalization theorem is assumed. agda (proof) agda (proof)

NativeTyping.agda[7] transcribes the original six typing rules and annotated beta equivalence, not a bidirectional checker. For a fixed closed type `A`, `NormalAt labels N d A t` asserts normality, exactly `N` nodes, original typing at `A`, and a sort rank of `A` at most `d`. Typing and rank certificates are propositionally truncated so that different derivations do not create additional inhabitants. The resulting typed family and its cardinal bound are checked directly:

```
-- labels names the fixed alphabet; A is an original closed annotated
↪  type.
-- Length is exactly N; d bounds the rank of A, not just redexes (there
↪  are none).
TypedNormal : ∀ {a} → (labels : Fin a → Nat) → (N d : Nat) → Term 0 → Set
TypedNormal {a} labels N d A = Σ (ExactNf a 0 N)
  (λ t → NormalAt labels N d A (erase (Fin a) (fst t)))

typed-normal-bound : ∀ {a} (labels : Fin a → Nat) (N d : Nat) (A : Term
↪  0)
  → a ≤ N → AtMost (TypedNormal labels N d A)
      (tetration (4 · N + 4) (d + 3))
typed-normal-bound {a} labels N d A p = subtype-finite
  (λ t → NormalAt labels N d A (erase (Fin a) (fst t)))
  (λ t → normalAt-isProp labels N d A (erase (Fin a) (fst t)))
  (closed-exact-normal-bound a d N p)
```

The constructor `pack-typed-normal` includes every originally typed normal term over the chosen alphabet. Here $a \le N$ is essential for the alphabet-independent numerical bound: take the labels occurring in an input of size `N`, not an arbitrarily large ambient alphabet. The dictionary `labels : Fin a → Nat` and type `A` are fixed parameters. Duplicate dictionary entries merely overcount representatives; they do not invalidate the bound. The parameter `d` bounds the type rank in this normal family; a normal term's redex bound alone would be vacuous. agda (proof)

7.4 Total typed apply-and-normalize

BoundedReduction.agda[8] implements capture-avoiding substitution, the greedy pass and its iteration on original annotated syntax. It proves forward beta reduction, decidable normality, and compatibility with replacing finite label indices by their original universe numbers. The size proof is unconditional: one pass has at most $2^{|t|}$ nodes, and every single-beta intermediate of d passes has at most $T_d(|t|)$, where $T_0(N) = N$ and $T_{d+1}(N) = 2^{T_d(N)}$. This seeded binary tower is bounded by $\max(2, N) \uparrow\uparrow (d+1)$. agda (proof) agda (proof)

In FiniteTrace.agda[9], a state is literally `State a n s = Fin (capacity a n s)`. Decoding recovers the annotated term; `state-bound` injects the closed state type, for $s = T_d(N)$, into

[7] agda/NativeTyping.agda
[8] agda/BoundedReduction.agda
[9] agda/FiniteTrace.agda

`Fin` of size $(4N + 4) \uparrow\uparrow (d + 3)$. The computed beta path has finite-indexed states and proposition-valued edge evidence:

```
data FiniteWalk (a n s : Nat) : State a n s → State a n s → Set where
  finished : ∀ {i} → FiniteWalk a n s i i
  move : ∀ {i j k} → IndexedStep {a} {n} {s} i j
    → FiniteWalk a n s j k → FiniteWalk a n s i k
```

Here `IndexedStep i j` is the propositional truncation of an original beta step between the decoded indices. `rounds-trace` computes this walk with the exact input and output endpoints; `walk-sound` proves forward reduction after decoding. Every state has the bound, not the type of arbitrarily long whole walks. agda (proof)

For two closed IIR normal forms, put `N = 1 + |f| + |y|`. FiniteApplication.agda[10] defines the successful result type and proves its entire cardinal bound, including proof-irrelevant reduction evidence:

```
-- The output is a bounded IIR normal form, not a raw term with an opaque
↪  tag.
-- Truncation counts represented terms, rather than their reduction
↪  proofs.
ApplicationResult : (d : Nat) (f y : Nf (Fin a) 0) → Set
ApplicationResult {a} d f y =
  Σ (SizedNf a 0 (tower d (inputSize f y)))
    (λ v → ‖ BoundedSteps (tower d (inputSize f y))
      (application f y) (erase (Fin a) (fst v)) ‖₁)

application-result-bound : (d : Nat) (f y : Nf (Fin a) 0)
  → a ≤ inputSize f y
  → AtMost (ApplicationResult d f y)
      (tetration (4 · inputSize f y + 4) (d + 3))
application-result-bound {a} d f y labelsBound =
  subtype-finite (λ v → ‖ BoundedSteps (tower d (inputSize f y))
      (application f y) (erase (Fin a) (fst v)) ‖₁)
    (λ _ → isPropPropTrunc)
    (closed-normal-bound a d (inputSize f y) labelsBound)
```

The raw function `applyWithinFinite` returns `Maybe ApplicationResult`: without typing premises, the selected number of passes need not normalize. Its successful result is an actual normal representative with a certified bounded forward reduction. Size cannot cause failure. agda (proof)

TypedApplication.agda[11] packages the original premises $f : \Pi x : X.Y$, $y : X$, and $\Pi x : X.Y : U_d$ in `TypedInputs`. The inputs f, y are IIR normal forms; no normalization or output-typing certificate is an input. `TypedResult` includes the exact original output typing at $Y[y/x]$, the bounded forward reduction, and the same cardinal bound. The executable interface is total:

```
-- Total application at the exact substituted type, with a bounded beta
↪  trace.
-- Rank descent and subject reduction are proved from the original rules.
applyTypedWithin : ∀ {a} (inputs : TypedInputs a) → TypedResult inputs
applyTypedWithin inputs = normal-result inputs , output-typed inputs
```

`normal-result` constructs the normal representative of `rounds d (app f y)`; `output-typed` certifies its original dependent type. Typing premises are propositionally truncated: they

[10] agda/FiniteApplication.agda
[11] agda/TypedApplication.agda

justify normality and output typing, but do not select the computed syntax. The entire result type injects into `Fin` of size $(4N + 4) \uparrow\uparrow (d + 3)$, with $N = 1 + |f| + |y|$. agda (proof)

The Agda proof uses the original six rules throughout. Binder-aware algebra gives weakening and typed substitution, including annotations. agda (proof) Parallel reduction proves confluence and uniqueness of normal forms. agda (proof) Typing inversion, context conversion and substitution prove type correctness and subject reduction. agda (proof) Type-rank uniqueness and substitution safety show that one greedy pass lowers a positive redex bound; bound zero implies normality. Hence d passes normalize, and normal arguments give the required application bound. agda (proof) Every single-beta intermediate also preserves the original rank bound. agda (proof) `normalize` packages these results for any originally typed, rank-bounded term, including its normal form, exact typing and bounded trace. It is idempotent and independent of the sufficient rank budget; two normalized inputs are beta-convertible exactly when their normal outputs are equal. agda (proof)

NativeRankCoverage.agda[12] computes a sufficient redex bound from every original typing derivation in a well-formed context, recursing through the annotated subject and using type-rank uniqueness. Thus `normalize-typed` requires neither a supplied rank bound nor a termination certificate.

The finite-indexed trace certifies the syntax-state bound. Its construction still evaluates unrestricted `Raw` internally; neither it nor the result bound asserts that every host type, proof object or runtime allocation is bounded. The following finite-table compilation makes the executable state interface explicit without claiming such a whole-runtime bound.

7.5 Church compilation at explicit ranks

Fix a nonempty label alphabet of size $a \le N$, and put $s = T_d(N)$ and $K = \mathrm{capacity}(a, 0, s)$, so $1 \le K \le (4N + 4) \uparrow\uparrow (d + 3)$. For every positive m and fixed rank r, define the original PTS terms

$$\begin{aligned} \mathsf{E}^r_m &= \Pi X : U_r.\ \Pi x_0 : X. \cdots \Pi x_{m-1} : X.\ X, \\ \mathsf{c}^{m,r}_i &= \lambda X : U_r.\ \lambda x_0 : X. \cdots \lambda x_{m-1} : X.\ x_i \qquad (0 \le i < m). \end{aligned}$$

Then $\mathsf{E}^r_m : U_{r+1}$ and $\mathsf{c}^{m,r}_i : \mathsf{E}^r_m$. Every inner arrow has rank $r + 1$; the outer product has rank $\max(1, r + 1) = r + 1$ because $U_r : U_0$. Both the type and its canonical selectors are normal and have exactly $2m + 3$ annotated nodes. These facts, including original typing for every index, are checked in ChurchFinite.agda[13]. agda (proof)

The finite injections already proved give the explicit encoding

$$\ulcorner t \urcorner_r = \mathsf{c}^{K,r}_{\iota(t)}, \qquad \iota(t) = \mathrm{encodeState}(t).$$

For the exact-length normal family one can use the smaller $K_N = \mathrm{capacity}(a, 0, N)$ instead. Typing/rank proofs select an external subfamily of these codes; they are not additional stored data. This encodes the represented elements of the Agda fibres, not their Cubical paths or an internal dependent induction principle.

For a finite table $F : \mathrm{Fin}(m) \to \mathrm{Fin}(n)$, define

[12] `agda/NativeRankCoverage.agda`
[13] `agda/ChurchFinite.agda`

$$\mathsf{Map}_r(F) = \lambda p : \mathsf{E}^r_m.\ \lambda X : U_r.\lambda y_0 : X.\cdots\lambda y_{n-1} : X.\ p\,X\,y_{F(0)}\cdots y_{F(m-1)},$$
$$\mathsf{Map}_r(F) : \mathsf{E}^r_m \to \mathsf{E}^r_n, \qquad (\mathsf{E}^r_m \to \mathsf{E}^r_n) : U_{r+2},$$
$$\mathsf{Map}_r(F)\,\mathsf{c}^{m,r}_i \longrightarrow^*_\beta \mathsf{c}^{n,r}_{F(i)}.$$

This uses the same answer type X throughout: it does not instantiate p at the higher-ranked type E^r_n, and requires no cumulativity. The original typing, displayed ranks and forward beta computation of selectors, unary maps and their finite iterations are checked directly against the native PTS rules.

For a binary table $F(i,j)$, abbreviate its rows by $R_i = q\,X\,y_{F(i,0)}\cdots y_{F(i,n-1)}$ and use

$$\mathsf{Map}^{(2)}_r(F) = \lambda p : \mathsf{E}^r_m.\lambda q : \mathsf{E}^r_n.\lambda X : U_r.$$
$$\lambda y_0 : X.\cdots\lambda y_{\ell-1} : X.\ p\,X\,R_0\cdots R_{m-1}.$$

Its type is $\mathsf{E}^r_m \to \mathsf{E}^r_n \to \mathsf{E}^r_\ell$, again of rank $r+2$, and it computes $F(i,j)$ on canonical inputs. Indeed each row has type X, and successive elimination of the two selectors chooses $y_{F(i,j)}$. This binary generalization is implemented and regression-tested in Agda; its all-index typing and computation theorems are not yet formalized. Consequently rank 1 suffices for the finite data and rank 2 for the operators, taking $r = 0$. The number of branches, hence program and type size, grows with the external bounds; this is not a uniform fixed-interface encoding of all elementary computations [8].

7.5.1 Compiling reduction passes

Define a total finite transition $g : \mathrm{Fin}(K) \to \mathrm{Fin}(K)$ by decoding a state, performing one greedy pass, and re-encoding when the result has at most s nodes; otherwise leave the state unchanged. Similarly let $b(i,j)$ encode the application of the decoded terms when it fits, and return i otherwise. These fallback branches make the tables total on the entire ambient carrier. They are never used on the intended bounded runs.

FiniteMachine.agda[14] implements these functions. Writing $t_{ij} = \mathrm{app}(\mathrm{decode}\,i, \mathrm{decode}\,j)$, it proves, without a typing premise,

$$|t| \le N \implies \mathrm{decode}(g^d(\iota(t))) = \mathrm{rounds}(d,t),$$
$$T_d(|t_{ij}|) \le s \implies \mathrm{decode}(g^d(b(i,j))) = \mathrm{rounds}(d,t_{ij}).$$

agda (proof)

FragmentMachineSoundness.agda[15] proves typed soundness of these same finite-state functions. `finite-normalize` decodes a normal form with the original type and rank bound; `finite-apply` does so at exactly $Y[y/x]$ for normal typed inputs. Both return forward reduction and bounded single-beta trace certificates. Their numerical budget premises exclude clipping, not assume normalization. agda (proof)

The original PTS programs are now explicit finite terms:

[14] `agda/FiniteMachine.agda`
[15] `agda/FragmentMachineSoundness.agda`

$$
\begin{aligned}
G &= \mathsf{Map}_r(g), & A &= \mathsf{Map}_r^{(2)}(b),\\
\mathsf{Norm}^r_{a,d,N} &= \lambda p : \mathsf{E}^r_K.\ G^d p & &: \mathsf{E}^r_K \to \mathsf{E}^r_K,\\
\mathsf{Apply}^r_{a,d,N} &= \lambda p : \mathsf{E}^r_K . \lambda q : \mathsf{E}^r_K.\ G^d(Apq) & &: \mathsf{E}^r_K \to \mathsf{E}^r_K \to \mathsf{E}^r_K.
\end{aligned}
$$

Here $G^d p$ is a metalevel abbreviation for exactly d nested applications, not an added recursion primitive. The compiler tabulates one pass and the application constructor, then emits these compositions; it does not fill a table with precomputed final normal forms. For a closed originally typed input t over the chosen alphabet, with $|t| \le N$ and RedexBound(ε, t, d), the decoded output is its normal form. For normal $f : \Pi x : X.Y$ and $y : X$, take d as the product's sort rank and $N = 1 + |f| + |y|$: the output decodes to a normal $z : Y[y/x]$ by the existing normalization and preservation theorems.

The following executable generators return literal original syntax (`Term 0`), not host functions standing in for PTS terms. Here `compileMap` and `compileMap2` implement the displayed tables, and `repeatMap` emits the d applications:

```
-- S bounds annotated source nodes; every target is a literal original
↪  PTS
-- term. The transition table computes ONE greedy pass, not normal
↪  answers.
passProgram : (a S r : Nat) → Term 0
passProgram a S r = compileMap r (greedy-state {a = a} {n = 0} {s = S})

applicationProgram : (a S r : Nat) → Term 0
applicationProgram a S r = compileMap2 r (application-state {a = a} {n =
↪  0} {s = S})

reductionProgram : (a d N r : Nat) → Term 0
reductionProgram a d N r = repeatMap r (capacity a 0 (tower d N)) d
  (passProgram a (tower d N) r)

applyReductionProgram : (a d N r : Nat) → Term 0
applyReductionProgram a d N r = lam E (lam (shift E)
  (app (shift (shift (reductionProgram a d N r)))
    (app (app (shift (shift (applicationProgram a S r)))
      (var (fsuc fzero))) (var fzero))))
  where
  S = tower d N
  E = choiceType r (capacity a 0 S)
```

This is code-to-code computation, distinct from the total typed Agda function above. It does not itself return an inhabitant of $Y[y/x]$ inside PTS or translate arbitrary Agda programs and quotient/truncation proofs. The generic native map proof and the Agda generators are separate formalizations, not a checked cross-language compiler-correctness bridge. Tabulation can be extremely large; no practical construction time or whole-machine memory bound is asserted. In particular, the source budget N does not bound the generated program or its execution: even one canonical state code has $2K + 3$ nodes.

7.6 Graded fragment soundness

For a fixed label dictionary, let $\mathcal{F}_s(A)$ contain closed IIR normal forms of at most s annotated nodes with propositional original typing at A. FragmentEmbeddings.agda[16] gives an injection

[16] `agda/FragmentEmbeddings.agda`

into `Fin (capacity a 0 s)`. Increasing the size budget gives injective embeddings satisfying identity and composition, preserving the decoded term and its typing. For these normal representatives,

$$\mathrm{decode}(u) =_\beta \mathrm{decode}(v) \quad \Longleftrightarrow \quad \mathrm{decode}(u) = \mathrm{decode}(v).$$

The right-hand side is literal equality of native annotated terms, not of finite codes when the dictionary repeats labels. Original type conversion acts on the typing fibre and commutes with the size embeddings. agda (proof)

Typed apply-and-normalize gives a graded application operation: for $f \in \mathcal{F}_s(\Pi x : X.Y)$ and $y \in \mathcal{F}_t(X)$ its output belongs to $\mathcal{F}_{T_d(1+s+t)}(Y[y/x])$, where $\Pi x : X.Y : U_d$. The API retains the alphabet premise $a \leq 1 + |f| + |y|$. The output decodes to the unique normal form of fy; subject reduction proves its exact type. Increasing budgets does not change the decoded result, and beta-convertible inputs give beta-convertible outputs. agda (proof) These are sound closed-term operations and coherent finite inclusions, not closure of one fixed carrier under every operation.

Alphabet inclusions preserve decoded terms and commute with normalization, size embeddings, conversion and application. Injective alphabet maps give injective carrier maps satisfying identity and composition. agda (proof) AlphabetCoverage.agda[17] constructs finite prefix alphabets for arbitrary annotated terms, contexts and valuations, with exact decoding. A prefix alphabet's size can exceed the number of syntax nodes; the general interpretation therefore includes alphabet size in its seed.

7.6.1 Contextual certificate soundness

An independent finite proof-tree grammar records all six typing rules and explicit beta instructions, including reduction inside annotations. `Accepted` recursively checks subtrees and local syntax equalities; `accepted?` decides this predicate without a native typing or normalization oracle. Acceptance implies original `Typ`, and every original derivation has an accepted certificate. This is checker adequacy, not a replacement typing judgment. agda (proof)

Context certificates check each declaration's sort; valuation certificates check every image against the substituted original context lookup. Their decidable acceptance predicates derive well-formedness and typed substitution. In particular, no native typing, well-formedness or normalization premise is hidden in the following input predicate: agda (proof)

```
record ModelInput (m n : Nat) : Set where
  field
    source : ContextCertificate m
    target : ContextCertificate n
    environment : ValuationCertificate m n
    derivation : Certificate m
open ModelInput public

InputAccepted : ∀ {m n} → ModelInput m n → Set
InputAccepted x = ContextAccepted (source x) × ContextAccepted (target x)
  × ValuationAccepted (certificate-context (source x))
      (certificate-context (target x)) (environment x)
  × Accepted (certificate-context (source x)) (derivation x)
```

[17] `agda/AlphabetCoverage.agda`

`model-soundness` interprets every accepted input, not merely closed terms or normal valuation images. It substitutes the environment into the subject and type, computes a common sufficient rank bound, and normalizes both:

```
ModelOutput : ∀ {m n} → ModelInput m n → Set
ModelOutput x = FiniteInterpretation to𝕟 (certificate-context (target x))
  (model-term-code x) (model-type-code x)

model-soundness : ∀ {m n} (x : ModelInput m n) → InputAccepted x →
↪  ModelOutput x
model-soundness x h = realize-typing to𝕟 (input-target-wf h)
  (model-term-code x) (model-type-code x)
  (subst2 (Typ (certificate-context (target x)))
    (sym (model-term-recovers x)) (sym (model-type-recovers x))
    ↪  (input-typing h))
```

`ModelOutput` contains their finite normal representatives and forward reductions, with original typing at the normalized type. Subject reduction also retains exactly the substituted source type. The algorithm chooses the alphabet and grades from the checked data; callers supply no guessed budget or desired output. `check-and-interpret` returns `nothing` only for rejected certificates, never because an accepted input failed to normalize. agda (proof)

The decoded interpretation respects conversion, is independent of typing witnesses and sufficient budgets, and commutes with valuation composition:

$$\mathrm{nf}\big(\mathrm{nf}(t[\sigma])[\tau]\big) = \mathrm{nf}\big(t[\tau \circ \sigma]\big).$$

Here valuations are arbitrary well-typed syntactic substitutions, not arbitrary set-valued assignments. NativeValuations.agda[18] proves identity, composition, context comprehension and substitution laws for the original annotated syntax; normalization preserves those equations. agda (proof)

Finiteness counts the context and type as data, not as unbounded external indices. FragmentUniverses.agda[19] bounds types, values and valuations; FragmentInterpretation.agda[20] constructs their jointly normalized representatives. Write $C_{a,n}(s) = \mathrm{capacity}(a, n, s)$ and $D_{a,n}(s) = \prod_{j<n} C_{a,j}(s)$. The joint context/type/value package $\mathcal{P}$ and context/substitution package $\mathcal{M}$ admit injections with bounds

$$|\mathcal{P}_{a,n,s,d}| \le D_{a,n}(s)\, C_{a,n}(s)^2,$$
$$|\mathcal{M}_{a,m,n,s}| \le D_{a,m}(s)\, D_{a,n}(s)\, C_{a,n}(s)^m.$$

Declarations retain their preceding scopes; all annotations and substitution images are counted, while semantic certificates are propositional fibres. Every typed valuation between well-formed contexts has such a package at computed bounds. Images may remain raw: independently normalizing the components of a dependent tuple is not assumed to preserve its exact type. These bounds concern the represented packages, not arbitrary proof trees or runtime allocations. agda (proof)

The interpretation is externally nontrivial. For $\mathsf{Void} = \Pi X : U_0.X$, canonical-form inversion rules out a normal inhabitant: a normal function must be a lambda, whose body would inhabit the arbitrary type variable X. In the context $X : U_0$, no normal neutral function or

[18] `agda/NativeValuations.agda`
[19] `agda/FragmentUniverses.agda`
[20] `agda/FragmentInterpretation.agda`

other normal constructor can do so. Total normalization gives `Void-empty` for every original closed typing derivation, hence `accepted-Void-rejected` and emptiness of every finite Void fibre. These are Agda metatheorems, not internal PTS consistency proofs. agda (proof)

7.7 Fragment interpretations and internal consistency

The result above is all-six-rule contextual soundness in a graded syntactic interpretation. Every fixed typed input has computed finite coverage, and larger grades cover more inputs. These graded results alone do not make each stage a **finite first-order model of a logical fragment**. That requires a total interpretation of the chosen signature on fixed finite domains, coherent dependent fibres, and all chosen axioms under every semantic valuation. Operations that increase grades do not automatically supply this closure. The explicitly guarded construction below supplies fixed finite domains; its axioms retain their bounds rather than asserting unrestricted closure. For example, iterating $A_0 = U_0$ and $A_{k+1} = \Pi x : U_0.A_k$ gives normal types of rank one for $k > 0$, using only label zero and $2k + 1$ nodes: a fixed size-bounded faithful type carrier cannot contain the whole family. This does not rule out nonfaithful finite models.

There is a sharper obstruction if raw computation is interpreted as equality in a total, untyped applicative algebra. On a finite carrier, the equation $(Kx)y = x$ makes $x \mapsto Kx$ injective, hence surjective. Choose a with $Ka = K$. Then $Kx = (Ka)x = a$ for every x, so injectivity makes the carrier a singleton. Any positive typing fact then contradicts an empty Void fibre. Neither extensionality nor another combinator is needed. This theorem does not apply to typed or resource-indexed application, or to MM0's directed `red` predicate, which is not equality. Finite models of reduction and partial interpretations are a possible alternative [11]; their dependent typing and conversion soundness for PTS-rk must still be established. A finite model of selected ground instances, constructed below, does not assert closure under all quantified instances.

Theorem 7.3 (The finite native-model boundary)**.** *Suppose a single term domain has total application, a compositional interpretation of closed terms, soundness for original closed typing and directed beta reduction, the Hilbert conversion and type-reduction rules, and an empty Void fibre. Then its interpretation of the k Church selectors is injective for every k. Consequently the domain is infinite.*

To distinguish two selectors, instantiate them at U_1 and choose branches between Void and the inhabited type E_1^0, both of sort U_1. Equal selector values would give one semantic test value reducing to both types. Conversion followed by type reduction would then transport an inhabitant into Void. No symmetry or antisymmetry of reduction is assumed. The finite version needs only these k selectors and their finitely many tests, and forces at least k semantic values. Thus this is not a prohibition on size-guarded fragments or growing graded carriers. It does rule out obtaining the full native soundness interface on one fixed finite carrier. The literal MM0 theory is identified with the independent catalogue below; extending this obstruction to it still requires the original-PTS completeness bridge.

The independent checker and its external soundness provide a precise starting point for internal reflection. Their translation into PTS remains open: equations on canonical Church codes do not give a finite-case or dependent induction principle for arbitrary inhabitants of E_K^r. Compiler correctness must connect the encoded checker and interpretation to the specified certificate predicate, including its conversion witnesses.

The intended endpoint is an original PTS proof of a specified consistency sentence, not a

metalevel assertion that no term of type Void exists. In the repository's consistency plan[21], this is $\vdash c : J(\mathcal{NAT}(C))$, where C says that the fixed predicate-only checker accepts no contradiction certificate, including all its trace witnesses. A promising route uses supplied tower resources to prove the checker's bounded soundness in elementary arithmetic, then Pakhomov's transfer for predicate-only Π_1 sentences [9]:

$$\mathsf{EA} \vdash C^{2\mathcal{S}} \quad \Longrightarrow \quad \mathsf{H} \vdash \mathcal{NAT}(C) \quad \Longrightarrow \quad \vdash c : J(\mathcal{NAT}(C)).$$

Here $\mathcal{S}$ is the superexponential cut; the last implication requires an actual interpretation J of the stated theory into PTS. The finite Church construction supplies computational ingredients, not those soundness and interpretation proofs. External results for every fixed budget are not one internal proof quantifying over all budgets. This distinction leaves the Pakhomov-style self-consistency objective plausible without identifying it with an unrestricted arithmetical consistency sentence. The external consistency needed by the ground models below is proved separately; it is not inferred from positive Horn satisfiability.

7.8 A locally finitely satisfiable target

There is a precise ground-query reduction, distinct from a faithful finite model of syntax. Ground the Hilbert inference rules, including the auxiliary syntax, substitution and reduction graphs. Introduce one nullary predicate p_J for each closed judgment J, and, for every ground rule instance, add

$$J_1, \ldots, J_k \Rightarrow J \quad \mapsto \quad (p_{J_1} \wedge \cdots \wedge p_{J_k}) \Rightarrow p_J.$$

Purely positive clauses allow the valuation making every atom true, including every alleged proof of Void. To exclude it, add the **denials**

$$p_{t:\mathsf{Void}} \Rightarrow \bot \qquad \text{for every closed subject } t.$$

More generally a denial forbids simultaneous truth of a finite list of atoms. A constrained model satisfies the positive clauses and makes every denial's premises false jointly. Thus every model, not merely the chosen least one, has an empty ground Void fibre. This is an effective Horn theory with constraints in a countably infinite recursive vocabulary; finite conversion certificates can be enumerated without deciding raw conversion.

Theorem 7.4 (Consistency-sensitive ground reduction)**.** *The constrained theory has a model exactly when no forbidden list is jointly derivable. Under this consistency condition, J is derivable exactly when every constrained model satisfies p_J. The least model interprets p_J as derivability and reflects every atom. Every finite ground subtheory has a one-element-domain constrained model.*

Soundness is induction on derivations; reflection uses the least model. Its satisfaction of the denials is the consistency obligation, not an automatic property of Horn clauses. Restricting the rules preserves this obligation. Conversely, every derivation uses finitely many rule instances, so satisfiability of all finite ground subtheories implies that no forbidden derivation exists. The singleton domain carries no syntax: independent nullary predicates retain the judgment distinctions. A singleton domain does not mean that these predicates are all true.

[21] `../self_consistency/README.tex`

For the checked Hilbert catalogue, grounding preserves and reflects its inductive derivations, and catalogue soundness transports them to original typing. External Void exclusion therefore supplies the required denials; the Agda native theorem above separately covers all six original typing rules. The Rocq bridge uses the same external canonical-form argument and original normalization, with no consistency premise. Original-PTS-to-catalogue completeness remains unformalized. The literal MM0/native catalogue correspondence, including its computational graphs, is established below; it does not supply that separate completeness proof.

7.8.1 Exact finite ground fragments

For a finite list F of positive ground clauses, compute

$$S_0 = \varnothing, \qquad S_{i+1} = S_i \cup \{J \mid (P \Rightarrow J) \in F,\ P \subseteq S_i\}.$$

Only clause heads can be added. Consequently $S_{|F|}$ is closed and has at most $|F|$ atoms, and

$$J \in S_{|F|} \quad \Longleftrightarrow \quad F \vdash J.$$

Testing each retained denial against this finite saturation decides whether a constrained model exists and constructs one when the test succeeds. Every constrained model excludes the denied atoms; the constructed model also reflects underivability. These are executable results, not an assumed normalization oracle.

Specializing this construction to literal closed PTS judgment pairs gives an unconditional model for every finite list of valid catalogue-rule instances. All ground Void judgments are false, including subjects outside the observed atom set. Saturation implies original typing and exactly reflects derivability within the selected fragment. The concrete models and universe/identity/application regressions are checked separately.

Arbitrary constrained models may still assign additional, nonderivable atoms true. To reflect a finite fragment completely on an observed finite atom set Q, add its **signed diagram**:

$$\{p_J \mid J \in Q \cap S_{|F|}\} \quad \cup \quad \{\neg p_J \mid J \in Q \setminus S_{|F|}\}.$$

Every model of this completion agrees with finite saturation on every propositional formula over Q, including negation and implication. The canonical finite model satisfies the completion exactly when the denial test succeeds. Completion deliberately adds negative information; it is not a claim that underivability entails negation in the uncompleted Horn theory. Its negatives are relative to F and need not survive adding more source rules. Neither saturation nor completion decides derivability in the infinite source theory.

These finite models also yield literal original-PTS programs. For a finite nonempty query list, Church table compilation gives a rank-$r+2$ map from its selector carrier to rank-$r+1$ Booleans. Applied to a canonical query code, `query_program` reduces to true exactly for F-derivable atoms and to false exactly for underivable ones. For consistent fragments, `formula_program` similarly computes completed-diagram truth; `denial_program` returns true exactly when the constrained fragment has a model. Typing and these forward-beta equivalences are checked, with confluence distinguishing the two normal Boolean outputs. Saturation and table generation here occur externally. The emitted PTS term evaluates the resulting finite table; it is not a uniform internal saturation algorithm or an internal proof of its own reflection theorem.

7.8.2 Checking all valuations in a fixed finite candidate

There is also a complete check for a specified finite candidate algebra. Fix a nonempty finite domain D and total interpretations of the constructor symbols. Represent a universally quantified positive or negative rule by its semantic template on D^k. Enumerate **all** vectors in D^k, not just values named by designated source terms, and saturate the resulting finite clauses. `check_candidate` succeeds exactly when some predicate interpretation satisfies every supplied template under every valuation; on success its least interpretation is such a model on the same fixed D. A denial of typing at Void therefore excludes every semantic subject, not merely the named ones. This semantic-grounding theorem is checked, including a regression whose violation occurs at the second domain element.

The actual MM0 signature is now represented by sorted expression trees: nine constructors, eleven predicates and all 55 named universal axioms. Selection retains whole axioms, not selected valuations. Every axiom and the added universal Void denial pass a proved sort/arity checker. Expression evaluation, semantic grounding and universal satisfaction agree; evaluation of well-scoped terms never uses an out-of-range-variable fallback. The transcription is independently compared with the expanded MM0 source; that textual checker is not a kernel-verified MM0 parser or native adequacy proof. If equality is added as a predicate, its actual finite diagram must still be imposed; saturation cannot choose its meaning freely. First-order substitution preserves sorting and commutes with evaluation. Independent source derivations instantiate the selected axioms with closed, sorted expressions; they contain no native typing premise. Their recursive grounding compiler produces finite semantic Horn proof trees, proving soundness under every interpretation satisfying the selected schemas.

7.8.3 Total constructor search and internal certificates

Interpret each of the three sorts on a separate copy of an m-element nonempty domain. Enumerating all operation tables gives exactly

$$m^{2+3m+3m^2+m^3}$$

candidates: the exponent counts the rows for two nullary, three unary, three binary and one ternary constructor. Every total constructor interpretation occurs pointwise. Lookup is total and every operation row is present, so the table fallback is never used. For each selected axiom list, exhaustive search returns a model exactly when one exists at these fixed sort cardinalities. Failure excludes all such interpretations. This is a complete decision procedure for the fixed domain, not an existence theorem for every fragment or an efficient search. The current specialization uses equal sort cardinalities.

A concrete two-element interpretation satisfies all valuations of the subtheory omitting `sveq` and `f0`, and has an empty Void fibre. These omissions are substantial: substitution is interpreted as identity, and the empty `form` relation disables positive catalogue introduction. The example tests genuine total operations and quantified satisfaction; it is not a model of full PTS-rk. The full axiom list is rejected on the one-element domain.

For internal certificates, interpret each semantic atom with truth value b by $\top = \Pi X : U_0.\, U_0$ when b is true, and by Void otherwise. A ground clause becomes its implication chain. Every valid clause has an explicit original-PTS proof: return a witness of $\top$, or a false premise when the conclusion is false. For all valuation instances $C_1, \ldots, C_q$, the generated finite satisfaction certificate has type

$$(C_1 \to \cdots \to C_q \to \mathsf{Void}) \to \mathsf{Void}$$

of rank at most four. Its closed inhabitability is equivalent to acceptance of the candidate checker and hence to existence of a predicate interpretation satisfying all selected axioms and denials under every valuation. The forward proof generator does not use canonicity; the external converse uses native Void exclusion. This is a finite continuation conjunction, not an unguarded quantifier over arbitrary Church inhabitants. Nor is this externally indexed family a single internal reflection theorem over all represented fragments. The finite native-model boundary above prevents silently extending these finite certificates to the full ungraded interface.

Deduction preservation is also proof-producing: a recursive compiler applies each realized clause to its compiled premises, yielding an original-PTS inhabitant of the independently interpreted conclusion. This covers source derivations of arbitrary finite depth, not just a bounded list of proof examples. For the literal MM0 schemas, successful constructor search supplies both the fixed-domain model and its native satisfaction certificate. The explicit two-element subtheory has an unconditional certificate and deduction translation: acceptance is proved from its constructor interpretation, not assumed.

7.8.4 Explicit size/depth-guarded quantified fragments

Fix a finite universe alphabet L, annotated-node budget N and syntax-depth budget D. At each required de Bruijn scope n, enumerate the original terms satisfying these bounds and adjoin an overflow value:

$$\mathcal{D}_n = \{\bot\} \sqcup \operatorname{Fin} |E_n|, \qquad E_n = \{t : \operatorname{Term}\ n \mid |t| \le N,\ \operatorname{depth}(t) \le D,\ \operatorname{labels}(t) \subseteq L\}.$$

The implementation stores enumeration positions, not membership proofs. Decoding an encoded admitted term returns that term. Its complete carrier enumeration has length at most $1 + 2^{2^{3D+n+|L|+4}}$. Application, product, annotated lambda and substitution are total: decode their arguments, perform the original constructor or substitution, and encode the result; failed decoding or an out-of-bounds result returns $\bot$. Thus a product consumes $\mathcal{D}_n \times \mathcal{D}_{n+1}$, and substitution consumes $\mathcal{D}_{n+1} \times \mathcal{D}_n$. The exact operation laws require the **output** bound as well as the input bounds. Neither guard mentions typing or the desired soundness conclusion.

Proof height is a separate parameter. Let $P_h(M, A)$ mean that the independent Hilbert rules derive $M : A$ using height at most h, with every displayed judgment inside the syntax bounds. For $h < H$, a guarded inference has form

$$\operatorname{Rule}_j(a_1, \ldots, a_j, a) \ \wedge \bigwedge_{i=1}^{j} P_h(a_i) \implies P_{h+1}(a), \qquad j \in \{0, 1, 2\}.$$

There are finitely many predicates $P_0, \ldots, P_H$ on the fixed finite judgment domain $\mathcal{D}_0^2$. The three rule graphs decode to the six original Hilbert inference forms; they do not assert that the conclusion is already derivable. Every semantic valuation is included. Overflow judgments are false. The stage increment is essential: even a one-premise rule on constant-size atoms can exceed a proof-height cutoff. Independent proof trees compute sufficient common size, depth and height bounds; every finite family of derivations has one common envelope.

The concrete model chooses $L = \{0, \dots, D\}$ and includes the maximum universe index in its depth measure. Here D is a syntax/label bound, **not type rank**. Its predicates exactly reflect bounded catalogue derivations, and all six quantified inference forms preserve them. Every true P_h judgment has its original native typing, and every semantic subject has an empty Void fibre. For $N \geq 3$ and $D \geq 2$, Void itself is represented without overflow; the universe judgment is also inhabited when $H \geq 1$. These are models of the stated guarded rules, not of the unrestricted theory. Catalogue derivations, and finite families of them, are covered by increasing the bounds; this does not supply the separate original-PTS-to-catalogue completeness theorem. This qualitative model bounds displayed judgments; its original beta/conversion side conditions do not yet bound their intermediate states.

Reduction witnesses can also be bounded explicitly. Enumerate **all** one-step original beta successors, including reductions in annotations. The Boolean graph $\mathrm{Reach}_{k,E}(t, u)$ checks a path of at most k steps with every state in the finite syntax list E; conversion checks two such paths to a common reduct in E. These checks are exact for the specified bounded paths and sound for original reduction/conversion. Every original finite path or conversion has a finite numerical envelope covering its intermediate states. Composition adds path budgets, $k + \ell$; it does not make a fixed cutoff transitively closed.

An executable version fixes a finite list of numerical catalogue instances and a path budget k. Enumerate every parameter tuple for **all six** Hilbert rules. Retain precisely the clauses whose displayed judgments fit and whose reduction/common-reduct checks pass. This does not omit a rule form or consult a typing oracle. Finite saturation gives the least predicate interpretation, with exact fragment-query reflection and an empty Void fibre. The Boolean rule graph and its all-valuation schema correspondence are proved, including the actual total application/product/substitution operations. Its original-PTS satisfaction certificate has rank at most four; arbitrary deductions in the fragment compile to native proofs. Here proof height is not truncated: saturation closes a finite set of guarded clauses. This differs from the P_h presentation above. The bounds control represented syntax and path states, not the host evaluator's entire working memory or the size of its generated certificate.

The executable fragments also cover the entire independent catalogue:

$$\mathsf{H} \vdash M : A \quad \Longleftrightarrow \quad \exists b \ \mathrm{query}_b(M, A) = \mathsf{true},$$

where b comprises syntax, label and path bounds and a finite catalogue list. Enlarging these data preserves truth. Every finite collection of catalogue proofs has one common budget, including all intermediate beta witnesses; true queries have faithful finite-carrier representations. This is external coverage and soundness. Void exclusion uses the proved native canonicity theorem; neither the generated certificates nor their external coverage constitute uniform internal reflection or an independent consistency proof of original PTS-rk.

For the literal MM0 syntax, a structural guard bounds every substituted metavariable value and every argument in every premise and conclusion. The whole source derivation computes such bounds. Guarded semantic schemas are compiled over **all** finite valuations; an inactive positive instance becomes a tautology, while universal Void denials remain unchanged. The finite checker is exact for this guarded theory, and accepted candidates produce the rank-at-most-four satisfaction certificates and deduction translations above. A separate exact-decoding law connects source guards to semantic guards; it contains no typing or consistency premise.

7.8.5 Literal MM0 adequacy and finite models

Quote each original annotated term into the MM0 constructor trees, retaining every de Bruijn index and universe label. Quotation is injective, has an exact scoped decoder, and commutes with the original renaming and substitution operations, including arbitrary binder cutoffs. The nine computational predicates—numerical comparison/maximum, lift, substitution, reduction, telescope formation, prefix and exposure—have their intended meanings **if and only if** their first-order-sorted ground instances have independent MM0 derivations. In particular, `form` computes syntax; it is not a typing oracle. A raw reduction starting at a quoted native term reconstructs a native reduction and a scoped endpoint; no endpoint-scoping premise is needed. Common-reduct conversion likewise agrees exactly with original beta conversion.

The 17 numerical axiom-row families generate exactly the 65 checked catalogue shapes. Exposure beyond the telescope length is rejected, and the substitution-family prefixes have lengths three through six. Every source `ax` derivation yields a catalogue entry, and every catalogue entry has an actual source derivation, for arbitrary numerical parameters. The six Hilbert inference rules then give

$$\mathsf{MM0} \vdash h(\ulcorner M \urcorner, \ulcorner A \urcorner) \quad \Longleftrightarrow \quad \mathsf{H} \vdash M : A.$$

For fixed N, D, enumerate the closed, first-order-sorted MM0 expressions of size at most N and depth at most D, and add overflow. The three signature sorts use copies of this fixed finite carrier. All nine constructors are total: malformed, wrong-sort or oversized results return overflow. The positive rule guard checks the declared sorts and bounds of **every** metavariable value and premise/conclusion argument. It includes every semantic valuation; failed guards make positive instances tautologies. The previously stated exact-decoding premise is proved for this concrete algebra. Here D measures constructor depth, not PTS type rank; unary numerals count towards size.

Finite Horn saturation reflects accepted queries into independent source derivations of their decoded atoms. Conversely, each source derivation has computed bounds at which its query is accepted and decodes exactly to the source atom. Thus the union of these finite fragments reflects the entire literal source theory, not merely selected proof examples.

Theorem 7.5 (Consistent finite models of all guarded MM0 schemas)**.** *For every $N \geq 5$ and $D \geq 3$, the concrete finite algebra admits a predicate interpretation satisfying all 55 size/depth/sorting-guarded MM0 axioms under every semantic valuation, together with the* ***unguarded*** *universal Void denial. Void is represented without overflow and the universe judgment is true. Any subset of the 55 schemas has the same existence guarantee. The resulting original-PTS satisfaction certificate's type has rank at most four.*

The proof reflects finite queries to MM0, reconstructs native catalogue derivations, and applies external native canonicity. No consistency or model existence premise remains. This establishes models of the explicitly guarded axioms, not the unrestricted theory. The generated certificates are externally indexed native terms, not uniform internal reflection over all represented fragments. Original-PTS-to-catalogue completeness and an independent internal consistency proof remain separate obligations.

7.8.6 Internal soundness for arbitrary fixed-fragment proofs

Fix a finite positive Horn theory $\mathcal{C}$ and query q. Give finite indices to the query and every atom in the clauses and denials. Encoding followed by decoding is the identity on this support; indexed derivations and Boolean queries agree exactly with their original versions. For guarded

MM0 fragments, the support includes every semantic valuation instance and every Void-denial atom, including overflow subjects.

Write its atom indices as $0, \dots, m-1$ and clauses as $c_j = (a_{j1}, \dots, a_{js_j} \Rightarrow b_j)$. Define

$$C_j(X) = X_{a_{j1}} \to \cdots \to X_{a_{js_j}} \to X_{b_j},$$
$$\mathsf{Proof}_{\mathcal{C}}(q) = \Pi X_0 : U_1 \cdots \Pi X_{m-1} : U_1.\, C_1(X) \to \cdots \to C_k(X) \to X_q.$$

This type encodes only the selected **positive inference rules**, not a truth table, a model, or an assumed soundness property. Its rank is at most three, independently of the number of atoms and rules. Every source proof has an explicit inhabitant. Clause introduction also accepts arbitrary encoded premise proofs, not merely outputs of that source compiler.

For a Boolean model v, let T_a be $\top$ or Void according to $v(a)$; both have rank one. Each valid clause has the explicit proof $h_j : C_j(T)$ constructed above. Consequently the literal original-PTS term

$$\mathsf{sound}_{\mathcal{C},v,q} = \lambda p : \mathsf{Proof}_{\mathcal{C}}(q).\, p\, T_0 \cdots T_{m-1}\, h_1 \cdots h_k : \mathsf{Proof}_{\mathcal{C}}(q) \to T_q$$

works for **every** inhabitant p of the independent encoding. Its type has rank at most four. This typing proof uses model validity, not native canonicity or induction over an externally supplied proof tree. Taking the least Boolean model and then using external Void exclusion proves the separate exactness statement

$$(\exists p \ \vdash p : \mathsf{Proof}_{\mathcal{C}}(q)) \quad \Longleftrightarrow \quad \mathcal{C} \vdash q.$$

For each fixed guarded MM0 fragment, the least-model validity premise is discharged. For $N \geq 5, D \geq 3$, every semantic Void query has an internal refuter $\mathsf{Proof}_{\mathcal{C}}(q) \to \mathsf{Void}$; denials are enforced by these refuters, not added as positive proof constructors. Source proofs compile to the independent encoding, and its finite-fragment union has the same exact source/catalogue coverage, retaining faithful query decoding.

7.8.7 Internally quantified queries and raw guard provenance

The query can also be internal. Fix the finite support once, using an arbitrary anchor atom, and put $Q_m = \mathsf{ChFin}_m(U_0)$. For an arbitrary $q : Q_m$, replace the separate propositions by a predicate $P : Q_m \to U_1$:

$$\mathsf{IProof}_{\mathcal{C}}(q) = \Pi P : (Q_m \to U_1).\, C_1(P\overline{0}, \dots, P\overline{m-1}) \to \cdots$$
$$\to C_k(P\overline{0}, \dots, P\overline{m-1}) \to Pq.$$

Here $\overline{i} : Q_m$ is the canonical selector. This independent family has rank three and contains no model or assumed soundness property. At each canonical query, explicit originally typed functions translate between this encoding and $\mathsf{Proof}_{\mathcal{C}}(i)$. Every source proof compiles, arbitrary encoded premises support clause introduction, and closed inhabitation is equivalent to finite derivability. The latter converse uses external canonicity; no mutual-inverse or parametricity principle is asserted.

The finite truth family is the actual PTS term

$$T_v = \lambda q : Q_m.\, q\, U_1\, T_0 \cdots T_{m-1} : Q_m \to U_1.$$

This dispatch is legal because $U_1 : U_0$ in the original PTS; it reduces to T_i at $\bar{i}$. Instantiating the motive with T_v and supplying the valid clause handlers gives a closed term

$$\mathsf{isound}_{\mathcal{C},v} : \Pi q : Q_m.\, \mathsf{IProof}_{\mathcal{C}}(q) \to T_v(q).$$

Its type has rank four. Both arguments are internal: its typing holds for arbitrary typed query and proof terms, including open ones, without assuming that the query is a selector. For a fixed guarded MM0 fragment, the least model discharges the model premise. Every supported canonical query agrees exactly with finite derivability, and every semantic Void query is refuted at $N \geq 5, D \geq 3$. Duplicate positions in the support are harmless for soundness; exact source correspondence uses the canonical index of each atom.

Raw instance provenance is checked independently. Its finite observations are either None (overflow) or $\mathsf{Some}(e)$ for an admitted closed constructor tree; overflow is not identified with zero. Direct tree evaluation, sorting and size/depth guards agree with every semantic valuation. Each active entry records a selected axiom and a sorted source substitution with exact arguments; an inactive entry is the tautology $a \Rightarrow a$. Induction on raw derivations reconstructs actual literal source derivations, so merging observations creates no spurious proofs. Every source derivation occurs at finite computed budgets. The indexed raw proof type also reflects every closed inhabitant to a literal source derivation, excludes overflow and Void proofs, and has exact source coverage over the union of finite budgets. These are the raw-query endpoints of `MM0IndexedReflection`, using the external inhabitation converse above.

The fragment, budgets and truth table remain external parameters. The internal constructions below realize fixed tables, not a recursive raw-tree interpreter or one budget-polymorphic PTS correctness theorem. The Void specialization still uses external canonicity; this is not an independent internal consistency proof. These ranks bound types, not generated program size or all normalization redexes.

7.8.8 Internal decoding and guarded computation

Proof translation can preserve an independent proof type, not just truth. If a finite atom map f sends every clause of $\mathcal{C}$ to a clause of $\mathcal{D}$, an explicit original-PTS term has type

$$\Pi q : Q_k.\, \mathsf{IProof}_{\mathcal{C}}(q) \to \mathsf{IProof}_{\mathcal{D}}(\mathsf{map}_f(q)).$$

Its type has rank four. Abstract the target predicate P and instantiate the source proof at $\lambda x.P(\mathsf{map}_f(x))$; mapped clause handlers then suffice. No selector canonicity is used, and clause order or duplicates do not matter. For the semantic-to-raw decoder, both supports are fixed before q is supplied. Exact guard provenance proves the required clause membership. Consequently arbitrary open queries and proofs translate internally to raw proofs at their decoded queries. Composition with raw least-model soundness also has type rank four.

Dependent finite case analysis needs a separate, property-independent generation certificate:

$$\mathsf{Enum}_r(q) = \Pi P : (Q_k \to U_r).\, P\overline{0} \to \cdots \to P\overline{k-1} \to Pq.$$

Each selector has an explicit constructor, and arbitrary inhabitants of this family eliminate dependently. Its rank is at most $\max(3, r+1)$; the uniform consumer's type has rank at most

$\max(4, r+2)$. No such certificate is assumed for an arbitrary **bare** Church code, and different motive ranks are not related by cumulativity.

Using Enum_3, pointwise equal finite-map programs have internal rank-one Leibniz equality $\mathsf{Eq}(x, y) = \Pi P : (Q_k \to U_1).Px \to Py$. Composition, identity and retraction laws are represented by terms whose types have rank at most five. This is a logical equality, not an assertion of native conversion for arbitrary open codes.

For raw guards, enumerate the selected axiom names with their complete valuation tuples, adding a rejected sentinel. Independently specify the finite graph by the facts $\varnothing \Rightarrow (i, b_i)$, where b_i is the structural raw guard. Both Boolean answers are retained in the atom support, but only the correct answer is a graph fact. A graph program g and Boolean projection π_b have internal certificates:

$$\begin{aligned}
&\mathsf{Enum}_1(q) \to \mathsf{IProof}_{\mathrm{graph}}(g(q)),\\
&\mathsf{Enum}_3(q) \to \mathsf{Eq}(\pi_i(g(q)), q),\\
&\mathsf{Enum}_3(q) \to \mathsf{Eq}(\pi_b(g(q)), \mathsf{guard}(q)).
\end{aligned}$$

The first consumer has type rank four; the equality consumers have type rank five. Their generation certificates are separate inputs, with explicit constructors at every canonical entry. At canonical entries, Boolean evaluation is exact; accepted graph proofs reconstruct a selected rule, sorted source substitution and all size/depth guards. Wrong answers have no closed graph proof, and rejected entries are inert. These last reconstruction and exclusion statements remain external and use the closed-inhabitation converse: the internal terms do not yet return raw source trees or substitution records.

8 The exact typed expansion quotient

Unlike the independent algebraic signature, this quotient is *relative to* native annotated terms Raw, applicative codes Code, expansion E, original typing Typ and beta equality Beta; Uraw(r) denotes U_r. The module Exact.agda[22] takes these data, not soundness or completeness proofs:

[22] `agda/Exact.agda`

```
data TyExact where
  ty : {r : Nat} (A : Raw) → Typ A (Uraw r) → TyExact r
  tyBeta : {r : Nat} (A B : Raw)
         (dA : Typ A (Uraw r)) (dB : Typ B (Uraw r))
         → Beta A B → ty A dA ≡ ty B dB
  setTyExact : {r : Nat} → isSet (TyExact r)

rankedSetExact : isSet RankedTyExact
rankedSetExact = isSetΣ isSetℕ (λ _ → setTyExact)

data TmExact where
  tm : {r : Nat} (A : Raw) (dA : Typ A (Uraw r))
     (p : Code) → Typ (expand p) A → TmExact
  tmBeta : {r : Nat} (A B : Raw)
         (dA : Typ A (Uraw r)) (dB : Typ B (Uraw r))
         (p q : Code) (h : Typ (expand p) A) (k : Typ (expand q) B)
         → Beta A B → Beta (expand p) (expand q)
         → tm A dA p h ≡ tm B dB q k
  setTmExact : isSet TmExact

tyOfExact (tm {r} A dA p h) = r , ty A dA
tyOfExact (tmBeta {r} A B dA dB p q h k e v i)
  = r , tyBeta A B dA dB e i
tyOfExact (setTmExact t u p q i j)
  = rankedSetExact (tyOfExact t) (tyOfExact u)
      (cong tyOfExact p) (cong tyOfExact q) i j
```

Both endpoints of a term path are originally typed; type equality is retained alongside equality of expansions. The common rank on each path matches uniqueness of native sorts. The fibre of tyOfExact recovers a specified ranked type without a term type index. Given native beta reflexivity, the beta constructors identify duplicate typing witnesses; set truncation makes equality witnesses irrelevant.

Ordinary inductive representatives with explicit typed beta relations are also defined in the same file. Their type-respect lemma is checked; the setoid laws follow when the supplied native beta relation is an equivalence. Deleting quotient paths without retaining those relations would instead give free syntax. Quotienting untyped codes first would be unsound as a typing representation: a beta class can contain untypable representatives.

All accompanying Agda modules are checked with Agda 2.8.0, Cubical v0.9 and the safe checker, without postulates, holes or disabled checks. This establishes well-formedness of the constructions, not an Agda-to-Rocq adequacy theorem or completeness of the dependent SK signature.

9 Future work

Beyond the second-paper programme in Section 1, the following three directions concern proposed systems, not changes to the original calculus or results established above.

1. **A quadratically checkable PTS-rk variant.** Apply the ideas of van den Berg and den Besten [10]: replace implicit definitional conversion by explicit propositional computation and transport witnesses. Develop an objective variant preserving the rank discipline, and prove deterministic worst-case $O(L^2)$ type checking, where L includes the context, term, type, numerical ranks and all annotations and witnesses. Their Theorem 7 proves quadratic checking for objective type theory; adapting it to PTS-rk is a separate obligation. This bounds checking, not proof search or normalization.

2. **A logic-free arithmetic of partial functions.** Develop a system analogous to logic-free primitive recursive arithmetic, with the following requirements.

   - Terms denote partial functions, rather than necessarily total functions. The basic judgment is extension, not equality:

   $$f \sqsubseteq g \quad \Longleftrightarrow \quad \forall \vec{x}, y \left(f(\vec{x}) \downarrow y \Rightarrow g(\vec{x}) \downarrow y \right).$$

     Thus the graph of f is included in the graph of g; symmetry is not an inference rule.

   - Every partial computable function must be expressible using a finite term-forming basis, in the spirit of finite bases for elementary recursive arithmetic. Computational completeness is a requirement on the intended standard interpretation, not on each finite model.

   - Use reflexivity, transitivity and substitution, together with a suitable induction rule. Substitution and induction must be sound for partiality and extension. Require a finite core of axioms and one infinite scheme exists_n, whose intended meaning is: either all numbers are equal, or there is an initial segment of at least n distinct natural numbers. This is a semantic specification to encode in the extension calculus, not an addition of unrestricted first-order logic.

   - Require local finite satisfiability: for every finite numeral k, the finite core together with all exists_n for $n \leq k$ has a finite model validating the inference rules. Noncollapsed finite models realizing the retained initial segments are a stronger, useful goal; the collapse disjunct must not be mistaken for such a construction.

   - Require deterministic worst-case $O(L^2)$ derivation checking, or better, in the full encoded derivation length, including numerals, substitutions and induction certificates. Checking must not decide semantic extension of arbitrary programs or execute potentially divergent computations.

3. **Interpretation of the type theory in the arithmetic.** Construct an effective, derivation-preserving interpretation of the variant in (1) inside the system in (2). Specify the translations of typing, explicit computation witnesses and contradiction, with sufficient reflection or model soundness to support the intended consistency application. This must go beyond constrained ground-query reduction: retain contradiction exclusion while interpreting the quantified signature and justifying the required internal reflection. Positive finite satisfiability alone does not exclude the translated contradiction.

10 References

1. M. W. Bunder and W. J. M. Dekkers, *Are there Hilbert-style Pure Type Systems?* Logical Methods in Computer Science **4**(1:1), 2008, pp. 1–19. doi:10.2168/LMCS-4(1:1)2008[23]. Open-access paper[24].

2. R. B. Jones, *The Lambda-calculus, Combinatory Logic, and Type Systems*, 1998–1999. Conceptual overview[25].

[23] `https://doi.org/10.2168/LMCS-4(1:1)2008`
[24] `https://lmcs.episciences.org/839/pdf`
[25] `https://www.rbjones.com/rbjpub/logic/cl/cl017.htm#LSKICube`

3. A. Kaposi, A. Kovács and T. Altenkirch, *Constructing Quotient Inductive-Inductive Types*, Proceedings of the ACM on Programming Languages **3**(POPL), Article 2, 2019. doi:10.1145/3290315[26]. Author manuscript[27].

4. T. Altenkirch and A. Kaposi, *Type Theory in Type Theory using Quotient Inductive Types*, POPL 2016, pp. 18–29. doi:10.1145/2837614.2837638[28]. Conference paper[29]; expanded draft[30], Sections 3.1–3.3.

5. L.-T. Chen, F. Nordvall Forsberg and T.-C. Tsai, *Can We Formalise Type Theory Intrinsically without Any Compromise? A Case Study in Cubical Agda*, CPP 2026. doi:10.1145/3779031.3779090[31]. Author paper[32]; Agda artifact[33], inspected at commit `8db0830`. Sections 3–5 develop the natural-model QIIRT method.

6. M. Carneiro, *Metamath Zero: Designing a Theorem Prover Prover*, CICM 2020, LNCS **12236**, pp. 71–88. doi:10.1007/978-3-030-53518-6_5[34]. MM0 specification[35].

7. S. Broda and L. Damas, *On combinatory complete sets of proper combinators*, Journal of Functional Programming **7**(6), 1997, pp. 593–612. doi:10.1017/S0956796897002888[36].

8. Original PTS-rk manuscript[37], sections labelled `complexity`, `rep-bound`, `uniform-elementary` and `non-uniform-rep`, and the checked normalization development[38]. See `BoundedNormalization`, `Runtime`, `EstimatorChecks` and `AutomaticBound`.

9. F. Pakhomov, *A weak set theory that proves its own consistency*, arXiv:1907.00877v2, 2019. Paper[39]. Lemma 3.11 and Corollary 5.3 give the predicate-only transfer results; Appendix A relates the theory to Robinson's R.

10. B. van den Berg and M. den Besten, *Quadratic type checking for objective type theory*, arXiv:2102.00905, 2021. Paper[40]. Sections 2–3, especially Theorem 7.

11. P. Selinger, *Order-incompleteness and finite lambda reduction models*, Theoretical Computer Science **309**, 2003, pp. 43–63. doi:10.1016/S0304-3975(02)00038-5[41]. Author manuscript[42], Section 4. Finite reduction models are distinct from equational models.

12. T. Altenkirch, A. Kaposi, A. Šinkarovs and T. Végh, *Combinatory Logic and Lambda Calculus Are Equal, Algebraically*, FSCD 2023, LIPIcs **260**, pp. 24:1–24:19. Open-access paper[43]. Sections 1 and 7 specify the extensional equivalence and its scope.

[26] `https://doi.org/10.1145/3290315`
[27] `https://akaposi.github.io/finitaryqiit.pdf`
[28] `https://doi.org/10.1145/2837614.2837638`
[29] `https://akaposi.github.io/tt-in-tt.pdf`
[30] `https://akaposi.github.io/paper_ttintt.pdf`
[31] `https://doi.org/10.1145/3779031.3779090`
[32] `https://fredriknf.com/papers/tt-qiirt_cpp2026.pdf`
[33] `https://github.com/L-TChen/TTasQIIRT`
[34] `https://doi.org/10.1007/978-3-030-53518-6_5`
[35] `https://github.com/digama0/mm0/blob/0d414c0bfdaaeb7fea571895127abc1fa5a3d956/mm0.md`
[36] `https://doi.org/10.1017/S0956796897002888`
[37] `../../latex/PTS-rk.tex`
[38] `../NormalizationComplexity/README.md`
[39] `https://arxiv.org/pdf/1907.00877v2`
[40] `https://arxiv.org/abs/2102.00905`
[41] `https://doi.org/10.1016/S0304-3975(02)00038-5`
[42] `https://www.mathstat.dal.ca/~selinger/papers/cmaa.pdf`
[43] `https://doi.org/10.4230/LIPIcs.FSCD.2023.24`

A Complete MM0 source

The following listing is read from the same file as the excerpt in Section 5.2. The highlighting follows MM0's declarations, comments and dollar-delimited math strings.

```
-- PTS-rk: the 65-shape Hilbert theory (MM0 specification, not a consistency proof).
-- All judgments mean finite Horn derivability. Variables below are metasyntactic.
delimiter $ ( ) $;
strict sort nat;
strict sort tm;
strict sort tel;
strict provable sort judgment;
term z: nat;
term s: nat > nat;
term u: nat > tm;
term v: nat > tm;
term app: tm > tm > tm;
term pi: tm > tm > tm;
term lam: tm > tm > tm;
term nil: tel;
term cons: tm > nat > tel > tel;
def one: nat = $ s z $;
def two: nat = $ s one $;
def three: nat = $ s two $;
def four: nat = $ s three $;
def six: nat = $ s (s four) $;
def v0: tm = $ v z $;
def v1: tm = $ v one $;
def v2: tm = $ v two $;
def v3: tm = $ v three $;
def v4: tm = $ v four $;

-- Numerical graphs on constructor numerals.
term le: nat > nat > judgment;
term lt: nat > nat > judgment;
term max: nat > nat > nat > judgment;
axiom le0 (n: nat): $ le z n $;
axiom les (a b: nat): $ le a b $ > $ le (s a) (s b) $;
axiom lt0 (n: nat): $ lt z (s n) $;
axiom lts (a b: nat): $ lt a b $ > $ lt (s a) (s b) $;
axiom max0 (n: nat): $ max z n n $;
axiom maxz (n: nat): $ max n z n $;
axiom maxs (a b c: nat): $ max a b c $ > $ max (s a) (s b) (s c) $;

-- lift k M N increments free indices at least k; sub k X M N removes index k.
term lift: nat > tm > tm > judgment;
term sub: nat > tm > tm > tm > judgment;
axiom lu (k a: nat): $ lift k (u a) (u a) $;
axiom lvlo (k i: nat): $ lt i k $ > $ lift k (v i) (v i) $;
axiom lvhi (k i: nat): $ le k i $ > $ lift k (v i) (v (s i)) $;
axiom la (k: nat) (f g x y: tm):
  $ lift k f g $ > $ lift k x y $ > $ lift k (app f x) (app g y) $;
axiom lp (k: nat) (A C B D: tm):
  $ lift k A C $ > $ lift (s k) B D $ > $ lift k (pi A B) (pi C D) $;
axiom ll (k: nat) (A C B D: tm):
  $ lift k A C $ > $ lift (s k) B D $ > $ lift k (lam A B) (lam C D) $;
axiom su (k a: nat) (X: tm): $ sub k X (u a) (u a) $;
axiom svlo (k i: nat) (X: tm): $ lt i k $ > $ sub k X (v i) (v i) $;
axiom sveq (k: nat) (X: tm): $ sub k X (v k) X $;
axiom svhi (k i: nat) (X: tm): $ le k i $ > $ sub k X (v (s i)) (v i) $;
axiom sa (k: nat) (X f g x y: tm):
  $ sub k X f g $ > $ sub k X x y $ > $ sub k X (app f x) (app g y) $;
axiom sp (k: nat) (X Y A C B D: tm):
```

```
  $ sub k X A C $ > $ lift z X Y $ > $ sub (s k) Y B D $ >
  $ sub k X (pi A B) (pi C D) $;
axiom sl (k: nat) (X Y A C B D: tm):
  $ sub k X A C $ > $ lift z X Y $ > $ sub (s k) Y B D $ >
  $ sub k X (lam A B) (lam C D) $;

-- Original annotated beta reduction, reflexive-transitive and contextual; no eta.
term red: tm > tm > judgment;
axiom rr (M: tm): $ red M M $;
axiom rt (M N P: tm): $ red M N $ > $ red N P $ > $ red M P $;
axiom rb (A B X N: tm): $ sub z X B N $ > $ red (app (lam A B) X) N $;
axiom ra (f g x y: tm): $ red f g $ > $ red x y $ > $ red (app f x) (app g y) $;
axiom rp (A C B D: tm): $ red A C $ > $ red B D $ > $ red (pi A B) (pi C D) $;
axiom rl (A C B D: tm): $ red A C $ > $ red B D $ > $ red (lam A B) (lam C D) $;

-- Fixed ranked telescopes; annotations are in their successive binder scopes.
def t1 (a: nat): tel = $ cons (u a) z nil $;
def ti (a: nat): tel = $ cons (u a) z (cons v0 a nil) $;
def tk (a b: nat): tel =
  $ cons (u a) z (cons (u b) z (cons v1 a (cons v1 b nil))) $;
def tp (a b: nat): tel = $ cons (u a) z (cons (pi v0 (u b)) (s a) nil) $;
def tpx (a b: nat): tel =
  $ cons (u a) z (cons (pi v0 (u b)) (s a) (cons v1 a nil)) $;
def ts (a b c q r t: nat): tel =
  $ cons (u a) z (cons (pi v0 (u b)) (s a)
    (cons (pi v1 (pi (app v1 v0) (u c))) q
    (cons (pi v2 (pi (app v2 v0) (app (app v2 v1) v0))) r
    (cons (pi v3 (app v3 v0)) t (cons v4 a nil))))) $;
term stel: nat > nat > nat > tel > judgment;
axiom st (a b c q r t: nat):
  $ max (s a) (s b) q $ > $ max q c r $ > $ max (s a) b t $ >
  $ stel a b c (ts a b c q r t) $;
term prefix: nat > tel > tel > judgment;
axiom p0 (L: tel): $ prefix z L nil $;
axiom ps (k a: nat) (A: tm) (L R: tel):
  $ prefix k L R $ > $ prefix (s k) (cons A a L) (cons A a R) $;

-- form e L D E M T folds L over D:E, exposing exactly e initial lambdas.
-- This is a syntax graph, NOT a typing judgment. Only the fixed rows below use it.
term form: nat > tel > tm > tm > tm > tm > judgment;
axiom f0 (D E: tm): $ form z nil D E D E $;
axiom fp (a b c: nat) (A D E M: tm) (L: tel):
  $ form z L D E M (u b) $ > $ max (s a) b c $ >
  $ form z (cons A a L) D E (pi A M) (u c) $;
axiom fl (e a: nat) (A D E M T: tm) (L: tel):
  $ form e L D E M T $ > $ form (s e) (cons A a L) D E (lam A M) (pi A T) $;

-- Exactly 65 named shapes (some numerical instances coincide); K13 is omitted.
term ax: tm > tm > judgment;
axiom coreI (a: nat) (M T: tm):
  $ form two (ti a) v0 v1 M T $ > $ ax M T $;
axiom coreK (a b: nat) (M T: tm):
  $ form four (tk a b) v1 v3 M T $ > $ ax M T $;
axiom coreS (a b c: nat) (L: tel) (M T: tm):
  $ stel a b c L $ >
  $ form six L (app (app v2 v0) (app v1 v0)) (app (app v3 v0) (app v1 v0)) M T $ >
  $ ax M T $;
axiom coreP (a b r: nat) (M T: tm):
  $ max (s a) b r $ > $ form two (tp a b) (pi v1 (app v1 v0)) (u r) M T $ > $ ax M T $;
axiom typeI (a e: nat) (M T: tm):
  $ form e (ti a) v1 (u a) M T $ > $ ax M T $;
axiom typeK (a b e: nat) (M T: tm):
```

```
  $ form e (tk a b) v3 (u a) M T $ > $ ax M T $;
axiom typeS (a b c e: nat) (L: tel) (M T: tm):
  $ stel a b c L $ > $ form e L (app (app v3 v0) (app v1 v0)) (u c) M T $ > $ ax M T $;
axiom exposeP (a b: nat) (M T: tm):
  $ form three (tpx a b) (app v1 v0) (u b) M T $ > $ ax M T $;
axiom sortP1 (a d e: nat) (M T: tm):
  $ form e (t1 a) (u d) (u z) M T $ > $ ax M T $;
axiom sortP2 (a b d e: nat) (M T: tm):
  $ form e (tp a b) (u d) (u z) M T $ > $ ax M T $;
axiom sortP3 (a b d e: nat) (M T: tm):
  $ form e (tpx a b) (u d) (u z) M T $ > $ ax M T $;
axiom sortI (a d e: nat) (M T: tm):
  $ form e (ti a) (u d) (u z) M T $ > $ ax M T $;
axiom sortK2p (a b d: nat) (L: tel) (M T: tm):
  $ prefix two (tk a b) L $ > $ form z L (u d) (u z) M T $ > $ ax M T $;
axiom sortK2l (a b d: nat) (L: tel) (M T: tm):
  $ prefix two (tk a b) L $ > $ form two L (u d) (u z) M T $ > $ ax M T $;
axiom sortK3 (a b d e: nat) (L: tel) (M T: tm):
  $ prefix three (tk a b) L $ > $ form e L (u d) (u z) M T $ > $ ax M T $;
axiom sortK4 (a b d e: nat) (M T: tm):
  $ form e (tk a b) (u d) (u z) M T $ > $ ax M T $;
axiom sortS (a b c d e k: nat) (L R: tel) (M T: tm):
  $ stel a b c L $ > $ prefix (s (s (s k))) L R $ >
  $ form e R (u d) (u z) M T $ > $ ax M T $;

-- BEGINHILBERT
term h: tm > tm > judgment;
axiom h_universe (i: nat): $ h (u i) (u z) $;
axiom h_scheme (M A: tm): $ ax M A $ > $ h M A $;
axiom h_apply (f x A B T: tm):
  $ h f (pi A B) $ > $ h x A $ > $ sub z x B T $ > $ h (app f x) T $;
axiom h_convert (M A B C: tm) (r: nat):
  $ h M A $ > $ h B (u r) $ > $ red A C $ > $ red B C $ > $ h M B $;
axiom h_subject (M N A: tm): $ h M A $ > $ red M N $ > $ h N A $;
axiom h_type (M A B: tm): $ h M A $ > $ red A B $ > $ h M B $;
-- ENDHILBERT
```